OPENKEDGE / TECHNICAL WHITEPAPER

KedgeFlow

A Programmable Control Plane for Consequential AI Agents

Separating Cognitive Generation from Authoritative Execution

The Core Principle

An agent's proposal describes a possible action; a model's judgment is candidate evidence about it. Neither is, by itself, authority to mutate protected state.

Architecture and Deployment Profile

Version 0.9 | 6 October 2026 | Proposed Specification

Jun He and Deying Yu | OpenKedge Research

Copyright (C) 2026 OpenKedge LLC

Abstract

KedgeFlow specifies a programmable control plane for consequential agent actions: it checks a proposed action against policy and evidence, issues bounded permission to execute it, enforces that permission at the protected resource, and tracks the outcome. Its formal description, a **Distributed Epistemic Admission and Transaction Control Fabric**, emphasizes evidence-based decisions and coordinated execution. Nine logical components and six interfaces connect existing agent harnesses to these responsibilities.

The deployment profile binds user authorization to an authenticated agent workload and proof key. The Secure Production Identity Framework for Everyone (SPIFFE) identifies workloads; Demonstrating Proof of Possession (DPoP) binds access tokens to keys. The gateway records consumption of execution authority before obtaining target credentials. Human review is bound to the exact action, and Consequential Tool Descriptors (CTDs) record each tool's scope and guarantees. Negotiated Model Context Protocol (MCP) and Agent2Agent (A2A) extensions connect existing workflows. Cedar, etcd, and durable workflow engines supply reusable implementation mechanisms. Resource guarantees depend on the conditions the target can enforce and the outcomes it can establish. Complete schemas and bounded reference implementations accompany the specification. Production conformance and industry ratification remain separate requirements.

Reader's Guide

The Core Contribution

KedgeFlow defines components, ownership boundaries, and programmable contracts for a complete consequential-action control plane. It exposes absent, implicit, incomplete, and unverified responsibilities, and provides a stable map for categorizing existing and emerging tools.

For Buyers: Architecture, Risk, and Governance. KedgeFlow sits alongside local or managed agent workflows, with enforcement at the protected resource boundary. Models, planning, memory, and orchestration remain in the Agent Plane; backing target mutation privilege remains behind the Execution Gateway, under an independently enforced privilege boundary. It binds the exact approved action, valid evidence, concurrent-writer constraints, single-use execution allowances, and observed outcomes. Buyers gain an architectural blueprint for vendor-independent governance, replaceable integrations, and verifiable audit records across declared paths, identities, writers, target guarantees, and failure-recovery rules.

For Implementers: Components, Interfaces, and Substrates. The reference roles are **Policy Authority**, **Resource Registry**, **Admission Controller**, **Grant Authority**, **Execution Gateway**, **Resource Adapter**, **Outcome Service**, **Agent Adapter**, and **Evidence Provider**. Products can implement several roles. Existing services supply substrates: reusable mechanisms such as policy evaluation, storage, and workflow persistence. The Governance, Runtime Admission, Enforcement Control, Execution, Evidence, and Outcome Interfaces define explicit permissions and bound records. The `KF-JSON-0.2` profile specifies how those records are encoded and signed. Deployment work establishes identity trust, credential exchange, exclusion of obsolete execution authority, coverage of competing writers, and recovery from unknown outcomes. Existing standards supply primitives; KedgeFlow identifies how they must fit together.

Contents

Abstract i

Reader's Guide i

1. Introduction: The Missing Contracts Between Agent Tools 1

2. Wiring KedgeFlow to Existing Agent Workflows 4

3. Architecture: Planes, Control Boundaries, and OpenFlow 5

4. The KedgeFlow Component Model 9

5. A Component Map for a Changing Ecosystem 14

6. Programmable Interface Contracts 23

7. Execution Authority and Resource Guarantees 30

8. Implementation Blueprint and Worked Trace 37

9. Adoption, Ownership, and Deployment Gates 44

10. Acceptance Criteria, Evidence, and Release Gates 52

A. Conditional Enforcement Invariants 58

B. Durable State and Atomicity Boundaries 59

C. Identity, Review, and Saga Invariants 60

D. Complete Machine-Readable Contracts 61

E. Glossary of Architectural Terms 63

Document Status 64

1. Introduction: The Missing Contracts Between Agent Tools

An agent harness coordinates model calls, memory, planning, tools, and task execution. A consequential-action control plane governs whether a proposed external effect may proceed, under which conditions, and with what evidence of completion. The two responsibilities can exist in the same product, but their interfaces, privileges, and failure behavior must still be defined. KedgeFlow provides that definition as a Distributed Epistemic Admission and Transaction Control Fabric: a control plane for evidence-conditioned, coordinated effects across heterogeneous agent workflows.

A recurring failure is authority confusion: untrusted context induces an agent to misuse inherited credentials, privileged sockets, or remote tool sessions. Existing permissions, sandboxes, and remote MCP authorization can constrain this access. KedgeFlow asks whether every protected effect crosses independently enforced admission and exact-action authority. Section 8.2 applies this contract to a governed loopback gateway and its required OS boundaries.

Consider two agents sharing a three-node service that requires two available nodes. One proposes draining orders-db node 1; another proposes removing node 2. Both plans appear acceptable against the same observed capacity, even if reviewers approve them; executing both leaves one node. Evidence can also become stale before the effect. The required contract coordinates the shared capacity condition and binds human and workload identities to the exact action. A reservation sets aside the one available maintenance slot before physical draining; an unknown drain keeps that reservation. If a later workflow step fails, a confirmed earlier effect may require compensation: a separately authorized action that reverses or mitigates it.

Existing controls supply much of the solution: policy decision/enforcement points, scoped capabilities, durable workflows, conditional resource writes, and framework approval hooks [1, 2, 3]. Their composition must still identify who freezes the action, verifies evidence, issues authority, holds target credentials, arbitrates concurrent effects, and resolves ambiguous outcomes. Completeness must be assessed across that composition.

1.1. The Evidence-to-Authority Boundary

Definition 1 (Evidence-to-Authority Boundary)**.** The Evidence-to-Authority Boundary is the contract separating claims about a proposed action from permission to execute it. An Admission Controller evaluates a frozen proposal—an action whose typed parameters and identity bindings have been fixed—against policy and acceptable evidence. A Grant Authority binds the admitted proposal to scoped authority. An Execution Gateway validates that authority and mediates the protected effect under the Resource Adapter's declared guarantees.

A model score is evidence only for the question it evaluated. A human approval is evidence only for the action and context actually reviewed. A policy decision is effective only where enforcement binds it to the actual request. An execution receipt is evidence only for the attempt or outcome its source can establish. Keeping these meanings separate prevents an apparently successful stage from silently standing in for the next one.

1.2. Follow One Protected Action

The following sequence introduces the records used throughout the paper:

1. **Propose.** The Agent Adapter freezes the exact action as proposal q, including the human subject u, the requesting workload w, parameters, and target identity. A resource's *incarnation* identifies the particular instance; deleting and recreating it under the same name changes that identity.
2. **Admit.** The Admission Controller evaluates q under versioned policy Π_v. An *obligation* is a policy-required check; a *witness* is an evidence record offered to satisfy it. Accepted evidence discharges the obligation. Successful admission produces certificate C_q, recording what was checked and the approved constraints. Missing evidence or denial produces no certificate.
3. **Authorize.** The Grant Authority validates C_q and issues execution grant G for the exact action and intended gateway, called its *audience*. A shared durable record, the *authority ledger*, coordinates allowances and any required capacity or budget reservations. Admission and grant issuance are separate decisions.
4. **Claim and dispatch.** The Execution Gateway verifies q with G and atomically assigns the allowance to one attempt: the *claim*. It durably records the intended request as a *dispatch intent*, then obtains target credentials and calls the Resource Adapter. The adapter applies the declared *guard*: a predicate, or condition, over resource state. The target's supported mechanism determines the strength of that check. A durable claim prevents another redemption; it does not establish that the target effect occurred.
5. **Resolve the outcome.** A receipt records the attempt and the evidence available about its effect. The Outcome Service *reconciles* uncertain attempts by querying authoritative status. A lost response leaves the outcome unknown and the allowance consumed until the required resolution rules are met.

We write $H(x)$ for the cryptographic digest of a record or value x: a compact binding to the exact bytes specified by its contract. Section 6.9 fixes the wire-profile digests; Section 7.6 specifies the review-byte hashes. In diagrams, $q + G$ means the proposal accompanied by its grant, not arithmetic addition.

The Unit of Control

KedgeFlow controls a typed proposed action against a declared resource scope. A proposal is not authority; an admission certificate is not an execution grant; dispatch acceptance is not proof of committed effect.

1.3. The Architectural Contribution

The contribution is a compositional blueprint: **named responsibilities, explicit interfaces, and deployment completeness criteria**. It has four parts:

1. **A stable component vocabulary and descriptor profile.** Nine logical roles separate governance, context, admission, authority, enforcement, adaptation, evidence, and outcome recovery. Their definitions specify ownership and privilege boundaries rather than product categories.
2. **Programmable contracts.** Agent and control-plane implementations exchange versioned proposals, observations, admission results, grants, policies, and receipts through named interfaces. The OpenFlow comparison explains why these contracts matter across independent implementations.

3. **An ecosystem and gap map.** Existing tools are assigned to the responsibilities they actually supply. Empty or incomplete cells identify integration work, and the same questions can classify a new tool without renaming the architecture.
4. **A completeness test.** A deployment must demonstrate covered mutation paths, exact action binding, acceptable evidence and policy freshness, authority conservation, declared commit guarantees, and recovery from uncertain effects.

KedgeFlow proposes this combination and its contracts, without claiming invention of its individual mechanisms. Section 5.2 credits overlapping approaches and reusable implementations. A complete policy decision point (PDP) and policy enforcement point (PEP) deployment can realize the same boundaries; comparative advantage requires implementation and measurement.

1.4. What the Blueprint Covers

The paper focuses on mutations to a declared protected surface, such as changing a service allocation, modifying an identity policy, or writing a business record. The surface includes every credential and execution path capable of those effects. *Complete mediation* means that every in-scope path crosses the declared enforcement boundary. Read operations, disclosure, generated messages, and model training require their own coverage and policy profiles if they are consequential. They are not covered automatically by protecting a mutation API. A read-only tool classification therefore does not exempt a tool from confidentiality or caller-authorization policy (Section 4.6).

The design requires trusted administrators and issuers, effective mediation, durable authority coordination, and accurate resource declarations. It constrains authority but cannot correct faulty policy or false observations. The paper locates these conditions so buyers can assess a system and implementers can complete it.

1.5. Normative Scope of This Revision

MUST, **MUST NOT**, and **SHOULD** state requirements of the proposed deployment profile. A *profile* selects concrete identity, evidence, encoding, or execution rules for an implementation. The schema and reference artifacts make its contracts reviewable and executable; production coverage depends on the deployed identity, issuer, ledger, adapter, and target. A specification cannot supply a missing resource transaction primitive. MCP/A2A extensions require explicit negotiation.

2. Wiring KedgeFlow to Existing Agent Workflows

Figure 1 places KedgeFlow beside an existing workflow. The harness—whether a command-line interface (CLI), software development kit (SDK), or graph runtime—owns models, memory, and planning. For protected targets like identity and access management (IAM) systems, the Agent Adapter submits tool proposals, the control plane supplies authority, and the gateway and resource adapter mediate effects.

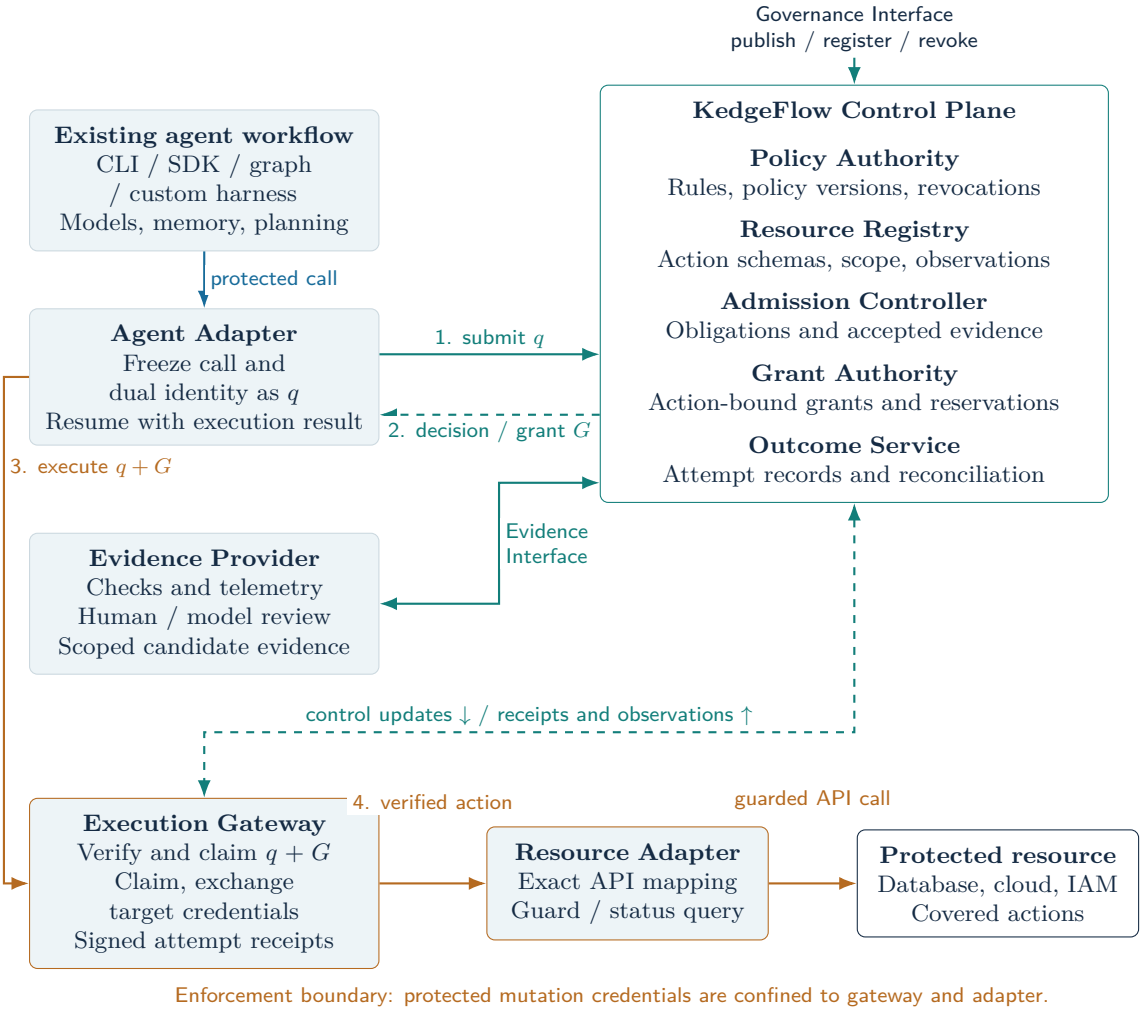


Figure 1: Wiring the control plane to an existing workflow. Solid arrows carry proposals, evidence, or effects; dashed arrows carry authority, control updates, or outcome records. The proposal binds human delegation and workload identity. After claim, the gateway exchanges narrowly scoped target credentials and reports signed receipts and authorized resource observations; the adapter supplies target evidence inside the enforcement boundary. Return results follow the same Execution Interface.

These are logical roles and do not require cloud placement. A Governed Loopback Gateway can host enforcement on the workstation while the control plane remains independently protected locally or remotely (Section 8.2). Native runtimes may implement the Agent Adapter contract directly; a proxy or hook can provide an attachment point. The gateway and Grant Authority also share the atomic authority ledger specified in Section 7.2. Complete mediation requires target controls to reject bypassing credentials and paths. Section 6 defines the six interfaces and the obligations at each endpoint.

3. Architecture: Planes, Control Boundaries, and OpenFlow

KedgeFlow separates three responsibilities: proposing an action, deciding and issuing its authority, and enforcing its effect. The **Agent Plane**, **Control Plane**, and **Enforcement Plane** name these responsibilities. They are logical domains; a deployment must additionally identify the processes, credentials, and administrators that create real trust boundaries.

3.1. The Three Planes

Agent Plane. Existing agent harnesses own prompting, models, memory, planning, collaboration, and workflow execution. The Agent Adapter freezes protected tool calls into typed proposals and routes results back to the workflow. On the declared protected surface, this plane has no usable backing mutation credentials. Its authenticated SPIFFE workload identity identifies the agent; an independently validated user delegation identifies the human subject and permitted delegation scope. Neither is a backing target credential.

Control Plane. Policy Authority, Resource Registry, Admission Controller, Grant Authority, and Outcome Service maintain policy, context, admission decisions, authority lifecycle, and effect records. Evidence Providers contribute scoped observations and judgments through the Evidence Interface. The controller can be replicated or federated, but authority ownership and coordination must be explicit for each scope.

Enforcement Plane. Execution Gateways and Resource Adapters authenticate execution requests, verify exact action binding, claim the execution allowance, and call backing APIs. The target's actual primitives determine whether guards can be atomic and whether completion can be established. Target credentials are obtained or retained only inside this boundary; OAuth token exchange must preserve the human subject and workload actor rather than interpreting a workload SPIFFE Verifiable Identity Document (SVID) as human consent. The protected credential is confined to this boundary; a sidecar sharing the agent's effective privilege domain does not create independent enforcement merely by being called a gateway.

The control plane's view is logically coordinated, not omniscient. Observations carry scope, source, age, and uncertainty. A registry record is neither a resource lock nor proof of live state. The design must declare who may write the target, which predicates need coordination, and where an authoritative reservation or commit guard is enforced.

Authority versions and recovery. An *execution root* is the durable record that owns a bounded allowance for an operation or workflow. Its *generation* identifies the current authorization version. An executable leaf root bounds one action; a composite root allocates allowance among child roots. A *saga* is a workflow of separately committed steps with compensating actions for supported recovery. Its composite root coordinates forward and recovery budgets (Section 7.7).

A *policy epoch* identifies the active policy version for a scope; an *ownership epoch* identifies the current authority owner. They let admission and enforcement reject obsolete authority. *Fencing* rejects operations carrying obsolete authority values at the declared enforcement point. A target-side fence must be checked by the target; changing a ledger record alone cannot cancel already accepted work. Section 7.2 specifies how generations, fences, reservations, and claims are coordinated.

Three Resource Adapter guarantee levels. A *predicate* is a condition over the declared resource state; the guard is the predicate required for an action. The *commit point* is the adapter-declared boundary at which the protected mutation takes effect. The adapter's level states how checking the guard relates to that effect:

Level A: Atomic predicate enforcement. The target checks the entire declared guard and commits the mutation indivisibly. A false guard prevents that mutation.

Level B: Pre-dispatch revalidation. The gateway checks current state before dispatch, but the target does not combine that check atomically with mutation. A concurrent change can invalidate the guard before the effect.

Level C: Deferred-effect observation. The target accepts work whose effect occurs later. The adapter tracks authoritative outcome evidence; later observation does not establish that the guard held at the effect.

These levels apply to an action path and predicate scope, not a product as a whole. They also differ from the *completion mode*, which specifies validity requirements at dispatch or commit. Section 7.4 gives the full level contracts, and Section 7.5 defines the modes. *Idempotency* means that repeating the same logical operation does not add another effect; it is a separate target property, not a consequence of a single-use grant.

3.2. What the OpenFlow Analogy Contributes

Software-defined networking (SDN) separates network control from forwarding. OpenFlow specifies a programmable interface between a controller and a switch's forwarding functions [4]. A controller can discover capabilities, inspect state, receive events, and modify forwarding behavior. The architectural lesson is an explicit controller-to-enforcement contract that supports independently implemented components. OpenFlow's switch specification does not prescribe a universal northbound business-policy API or eliminate all local control; it includes hybrid switches.

The network example below uses media access control (MAC) addresses alongside familiar IP, TCP, and UDP packet headers.

Responsibility	OpenFlow / SDN	KedgeFlow Correspondence
Controller inputs	Network applications, topology and events	Organizational policy, resource observations, evidence, and proposed actions
Programmable enforcement	Controller installs forwarding entries	Controller distributes policy, revocations, and action-bound authority to gateways
Enforcement element	Switch validates and applies supported match/action behavior	Gateway validates a grant; adapter enforces the supported resource guard and effect
Capability discovery	Supported tables, fields, actions and ports	Supported action schemas, covered resources, guard scope, adapter level and completion mode
State and outcomes	Statistics and datapath events	Observations, attempt receipts, authoritative outcome or explicit unknown
Separate submission path	Traffic enters the forwarding system	Agent submits a proposal to admission and later requests gateway execution

Table 1: An architectural correspondence, not protocol equivalence. The closest analogue to OpenFlow's southbound interface is KedgeFlow's Enforcement Control Interface.

Figure 2 visualizes this correspondence alongside the consequential differences where the analogy ends. An OpenFlow entry matches packet fields and applies supported actions. A KedgeFlow grant binds an exact proposed effect to a principal, audience, policy, evidence, validity interval, and declared resource guard. This is a useful comparison for programmable authority, but a grant additionally needs admission and durable consumption semantics. Packet forwarding and consequential state mutation have different failure and completion models.

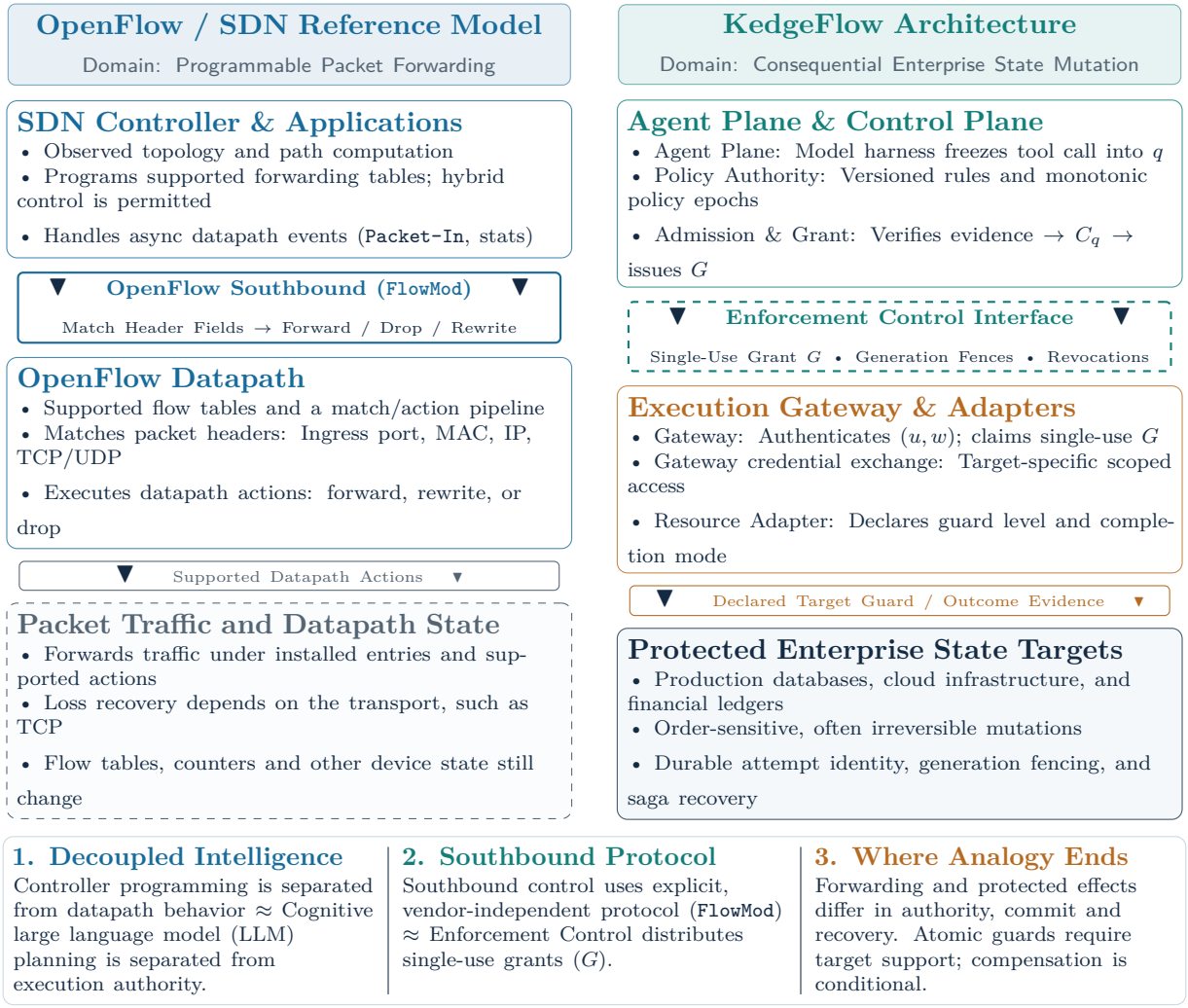


Figure 2: The OpenFlow / SDN correspondence and where the analogy ends. OpenFlow programs supported forwarding behavior through an explicit controller-to-datapath protocol (`FlowMod`). KedgeFlow separates cognitive planning in the Agent Plane from protected execution in the Enforcement Plane; the Control Plane decides and issues authority. KedgeFlow adds admission (C_q), single-use grants (G), declared target guards, and outcome reconciliation. Only a resource supporting atomic check-and-commit supplies the Level A guard contract.

KedgeFlow therefore names the contracts separately. The Runtime Admission Interface connects proposers to the controller. The Enforcement Control Interface connects controller and gateways. The Execution Interface redeems authority; the Evidence, Governance, and Outcome Interfaces complete the loop. Treating all of these as one southbound RPC list would hide caller permissions and implementation obligations. Section 6 defines both sides.

3.3. Where the OpenFlow Analogy Ends

OpenFlow's contribution is programmable control over supported forwarding behavior. It does not supply an agent transaction model. Three differences determine KedgeFlow's additional contracts.

Forwarding versus consequential mutation. A forwarding decision handles packet traffic; a protected action changes an enterprise resource and may be irreversible. Retransmission is a property of particular transports, not a guarantee of OpenFlow or every packet flow. Repeating a lost database write or payment request can duplicate an effect. KedgeFlow therefore needs durable attempt identity, resource idempotency, generation fencing, and explicit compensation. Network devices also maintain state; the useful distinction is the consequence and recovery contract, not a claim that switches are entirely stateless.

Duration and validity. A guarded SQL record update and a multi-minute infrastructure job expose different authority windows. These are illustrative timescales, not latency guarantees for either architecture. A dispatch-bound grant covers gateway initiation; a deadline required at target acceptance needs target enforcement. A commit-bound profile must maintain or revalidate its conditions at the actual effect. Token expiry or controller revocation does not automatically cancel work already accepted by a target.

Evidence decay and independent writers. Networking already has delayed and imperfect topology observations. Agent actions additionally depend on heterogeneous telemetry, human review, policy context, and writers that may not share a discovery protocol. A witness's age, provenance, question, and protected state projection determine whether it remains usable. A controller cannot infer current truth from a signature or from a recently read object version alone. The fabric combines evidence deadlines, authoritative reservations, and target checks; each supplies a different part of the temporal contract.

3.4. Observed State versus Authoritative Coordination

Return to the three-node example. Agents A and B can each observe capacity three and propose removing one node. The Resource Registry records that observation. The Admission Controller resolves the shared predicate available ≥ 2 and the operations that can affect it. The Grant Authority must coordinate reservations against that predicate in an authoritative ledger rather than merely reading the registry.

If A reserves one unit, B cannot reserve another against the same remaining allowance. A reservation record must bind its invariant scope, owner, action, generation, and release conditions. Claiming a grant validates its active generation and reservation in the same authority transaction. A consumed or uncertain attempt keeps its reservation until the effect is resolved; expiry alone cannot release capacity that a delayed operation may still remove.

This controller-side serialization protects the invariant only if all relevant writers participate and the resource honors the declared coordination or fencing contract. If an external administrator can concurrently remove capacity outside that contract, a registry and reservation store cannot guarantee the resource predicate. The Resource Adapter must provide a full-scope atomic check, a resource-enforced reservation/fence, or declare a weaker Level B/C guarantee. A conditional update to one node is not by itself an atomic guard over a multi-node availability predicate.

3.5. Deployment and Completeness

A minimal implementation may group policy, registry, admission, and grant functions in one controller service; place gateway and adapter in a separate privileged domain; and run outcome reconciliation as a worker. Co-location shares a compromise domain. Separate signing roles limit accidental authority mixing but do not prove fault independence.

A complete control plane is defined relative to its protected surface. It needs an owner for every required role, an implementation for every relevant interface, and evidence that the authority and outcome loop closes under faults. A diagram with every box populated is insufficient if an alternate credential path, stale-policy fallback, uncoordinated writer, or blind retry escapes the declared contract. Section 5.6 turns these boundaries into a gap register; Section 10 defines the validation evidence.

4. The KedgeFlow Component Model

A KedgeFlow component is a *logical responsibility with an interface, owned records, and a privilege boundary*. The nine names below are the reference vocabulary of this proposal. Use the full qualified name when citing a role, for example, *KedgeFlow Grant Authority*; use the shorter role name when context is clear. Acronyms are unnecessary to distinguish these responsibilities.

The names use established systems terms such as admission controller, authority, gateway, registry, adapter, and provider. They describe what a component does rather than a particular model, vendor, or implementation algorithm. A runtime, a protocol, a controller, and a database are different kinds of object: products and substrates implement roles; the roles define the contracts against which those implementations are assessed.

The catalog distinguishes responsibility from stored data and privilege. *Provenance* records where evidence came from and how it was produced. *Lineage* links a proposal to its admission, grant, attempt, and outcome. A *conflict domain* groups actions that can compete for a shared invariant, such as the maintenance capacity in Section 1. A *retirement barrier* prevents obsolete authority from being used; after possible dispatch, it must also exclude late effects before an allowance can be reused.

4.1. Roles and Ownership

Component	Responsibility	Owned Records	Authority Boundary
Policy Authority	Publish and activate policy; govern revocation and scope ownership	Policy versions, compatibility rules, ownership epochs, revocations	Policy signing; no backing mutation credential
Resource Registry	Resolve actors, actions, resources, coverage and observed dependencies	Identity bindings, action schemas, resource incarnations, provenance-tagged observations	Context supply; cannot grant authority or assert an atomic live snapshot
Admission Controller	Resolve obligations and verify evidence for a frozen proposal	Proposals, obligation manifests, discharge decisions, admission certificates	Admission signing; cannot mint execution grants
Grant Authority	Issue and conserve scoped execution authority	Issuance keys, roots, generations, reservations, transfers, retirement barriers	Grant signing; cannot execute backing mutations

Component	Responsibility	Owned Records	Authority Boundary
Execution Gateway	Validate, claim and mediate an exact authorized effect	Claims, attempts, durable dispatch intent, policy checkpoints	Backing credential custody and receipt signing
Resource Adapter	Map a verified action to the exact target API and guard	API mappings, target operation IDs, guard descriptors, reservation handles	Credential use only within gateway boundary; cannot issue authority
Outcome Service	Retain authorization/effect lineage and reconcile uncertain attempts	Receipts, retained objects, pending resolutions, outcome observations	Records and resolves; cannot reopen authority by itself
Agent Adapter	Connect a native runtime or tool hook to the protocol contracts	Tool/schema mappings, proposal correlations, pending workflow calls	Untrusted proposer integration; no backing mutation credential
Evidence Provider	Answer typed questions with scoped candidate evidence	Provider versions, provenance, question/result records	Evidence signing; cannot admit or grant merely by returning a positive result

Table 2: Nine citable logical roles. Ownership is a responsibility assignment, not a requirement for nine databases or independent services.

4.2. Control-Plane Components

Policy Authority. The authoritative publisher of permitted actions, assurance obligations, policy activation and compatibility, emergency revocations, and resource-scope ownership. Its methods `publish(Policy)`, `current(scope)`, and `revoke(Revocation)` return versioned authenticated records. Only authorized governance identities can change policy. Admission and enforcement use bounded-freshness distribution checkpoints; an expired policy view blocks new authority or dispatch.

Resource Registry. The resolver of authenticated actors, canonical action schemas, resource incarnations, gateway coverage, and observations/dependencies. Its methods `register(Descriptor)`, `resolve(ResourceRef)`, `observe(Observation)`, and `view(scope)` expose versioned context with explicit unknowns. A source timestamp or signature establishes the origin of an observation, not that it remains true at commit. The registry owns descriptors and observation history; authoritative reservations belong to the authority coordination ledger.

Admission Controller. The decision service that fixes a proposal's obligations under policy and checks the supplied evidence against them. Its `admit(Proposal, evidenceRefs)` returns `Admitted(certificate)`, `Denied(reasons)`, or `NeedsEvidence(unresolved)`. Only the first result supports grant issuance. Certificates bind the proposal, policy, obligations, accepted evidence, validity, and approved execution constraints. Unknown or expired evidence cannot discharge an obligation. Cognitive Admission Control [5] is one possible obligation-resolution method; the role is broader than that method.

Grant Authority. The issuer of exact execution grants, with `issue(Certificate, GrantRequest)`, `replace(gid, certificate)`, and `delegate(gid, childCertificate)`, where `gid` identifies a grant. It verifies admission, policy compatibility, requested scope, and execution budget. Repeated issuance for

the same admission/root returns the existing grant or status. Root generation, active-grant pointer, shared reservations, gateway claim, and transfer/retirement conditions use one coordinated atomic authority domain. Merely writing a new token cannot revoke an old token already claimed by a gateway. Claimed or uncertain effects block replacement until authoritative resolution and fencing permit it.

Outcome Service. The recorder and reconciler of authorization and effect lineage. Its `append`, `reconcile`, and `trace` methods retain the proposal, certificate, grant, attempt and outcome references under a declared retention policy. It resolves uncertain attempts through authenticated gateway/adapter or target queries. A gateway-signed receipt establishes the gateway's statement; durable target effect requires commit evidence of the strength declared by the adapter. Tamper evidence requires protected checkpoints or equivalent log integrity. An outcome resolution does not itself replenish a spent execution allowance.

4.3. Enforcement Components

Execution Gateway. The reference monitor exposing `execute`, `status`, and authorized `install` methods. It checks caller, audience, exact payload, policy/ownership epochs, evidence validity, active generation and completion constraints, then claims authority and records dispatch intent before invoking the resource adapter. All replicas share the coordinated claim contract. No claim quorum, valid authority, or required durable record means no new dispatch. The gateway holds the protected backing credential; IAM, network and application controls must deny bypass paths for the declared surface [6].

Resource Adapter. The trusted target integration with `describe()`, `dispatch(VerifiedAction, Guard, attemptId)`, and `query(attemptId)`. It specifies action-to-API mapping, resource incarnation, supported predicates, commit point, idempotency, late-effect fencing and completion evidence. `VerifiedAction` is created by the gateway inside the trusted boundary, not accepted as an agent flag. The adapter advertises one of the Levels A, B, or C introduced in Section 3.1 for each action and predicate scope. The detailed contracts are in Section 7.4; a brand or API family cannot establish the level. A general shell executor needs an additional constrained mapping to serve as an exact typed adapter.

4.4. Integration and Evidence Components

Agent Adapter. The runtime integration with `capabilities()`, `toProposal(toolCall)`, and `deliver(result)`. It maps protected calls to canonical proposals, authenticates the requesting workload, and preserves correlation while a workflow waits for admission or execution. A native runtime may implement the contract directly. An adapter reports unsupported hooks or alternate tool paths; it cannot remove credentials by assertion or supply the independent enforcement boundary.

Evidence Provider. A plug-in with `evaluate(Question, StateRef, ProposalRef)` returning a typed result with question version, proposal/state scope, source, observation time, validity, provenance and provider identity. Implementations can wrap deterministic checks, telemetry, human review, or model evaluation. A signature authenticates a statement, not its truth. The admission policy determines which provider classes may discharge each obligation; agreement among models does not by itself establish independence.

4.5. Roles, Interfaces, Stores, and Optional Strategies

The six interfaces in Section 6 specify exchanges between these roles. Stores support their durability and consistency: policy storage, registry/observation storage, an atomic authority/claim/reservation ledger, and outcome records. A transactional database or etcd can implement a substrate; it does not define action obligations merely by providing transactions.

Evidence acquisition scheduling is an optional strategy inside or alongside admission. Assurance-Aware Semantic Scheduling [7] can guide acquisition cost and order; direct provider calls are also valid. Quorum analysis, formal verification, identity continuity, and specialized approval workflows are additional strategies or evidence profiles. They do not add mandatory peer components, and their output must pass the same admission contract.

The National Institute of Standards and Technology (NIST) policy decision/enforcement separation provides an established point of comparison [1]. KedgeFlow makes admission, authority issuance, context and outcome ownership more explicit for this use case; it does not rename the NIST architecture or claim that every listed role is a distinct standardized NIST element. The component vocabulary remains useful when an implementation uses existing PDPs, policy administration systems, or capability services to fill several roles.

4.6. Consequential Tool Descriptors: CTD v1.0

The Resource Registry **MUST** register a versioned Consequential Tool Descriptor (CTD) before a tool can enter the governed execution path. A CTD binds the action schema, exact parameter interpretation, resource identity rules, consequence tier, adapter guarantees, conflict domain, and compensation policy. A *consequence tier* classifies the registered action's effects, from read-only to irreversible; it is distinct from the adapter's A/B/C guarantee level. The fields below define both classifications.

The Policy Authority separately binds admission and identity profiles to the signed action/descriptor reference. The signed descriptor digest is included in admission constraints and the grant; changing the descriptor requires new admission. Registry updates append a new version rather than overwriting a descriptor already referenced by authority.

CTD Field	Meaning	Required Validation
`consequence_tier`	Tier 0 `ReadOnly`; Tier 1 `IdempotentWrite`; Tier 2 `StateMutating`; Tier 3 `Irreversible`.	Classification is asserted under a registered action contract. Read-only can disclose information; idempotence does not imply reversibility, low consequence, or unrestricted access. Higher consequence requirements override a lower label.
`adapter_contract`	Level A/B/C, supported guard primitive, fence parameter, commit point, and completion modes.	The action path implements the declared primitive. A fence value is compared to target-maintained authority, not merely injected into an ignored argument. An async path declares eventual-effect and cancellation limitations.
`concurrency`	Canonical conflict-domain identifier, predicate descriptor, covered read/write set and writer set.	Predicates use a registered typed language, not arbitrary agent-supplied code. Shared invariants require complete participating-writer scope and resource-enforced coordination where claimed.

CTD Field	Meaning	Required Validation
`compensation`	Inverse or mitigating action descriptor, prerequisites, deadline and required budget.	A reversible action supplies a descriptor reference; an irreversible action explicitly declares no compensation. A compensator is a separately admitted effect, not an automatic exact inverse.
Rendering contract	Immutable template/version digest and approved field projection for human review.	Every execution-relevant parameter is rendered deterministically. Template integrity, schema compatibility and escaping are validated before review.
External governance binding	Policy-owned admission and identity profiles: obligations, provider classes and subject/workload trust.	Policy can strengthen a descriptor; neither a tool nor a runtime can lower the required assurance or substitute an untrusted identity.

Table 3: CTD v1.0 semantic requirements. Appendix D supplies the complete JSON schema, which rejects undeclared fields, and validated fixtures.

YAML is a human authoring representation. It **MUST** be parsed into the constrained JSON data model before RFC 8785 canonicalization, with duplicate keys, aliases, merge keys, implementation-specific tags, non-finite numbers and unsupported scalar conversions rejected. The reference profile signs canonical JSON, not raw YAML text. JSON for Linked Data (JSON-LD) interoperability requires a separately pinned context and expansion contract; remote context resolution is not accepted in the signed CTD profile.

4.7. Hardware-Bound Human Review as an Evidence Provider

A human-review Evidence Provider owns the verified review ceremony. It authenticates the reviewer, resolves the registered template, and renders the canonical action parameters and typed identifiers, consequence, and policy context. Agent-authored prose cannot replace, hide, or override those fields. The interface may show separately labeled explanatory context, but the authoritative review projection is deterministic. The reviewer must be able to inspect the complete bound action, including target incarnation, units, defaults and delegation actor.

The template is immutable by digest: revisions produce new versions and invalidate outstanding review challenges. The Evidence Provider prepares a one-use challenge bound to the proposal, active policy epoch, template/rendering digest, verdict, reviewer, expiry, and nonce. A *nonce* is a value whose one-time use is tracked to prevent replay. WebAuthn authenticates a response, called an *assertion*, to the provider's challenge using an enrolled public-key credential [8]. The provider verifies that assertion before publishing a signed witness. A click, screenshot, generic login assertion, or human message lacking this binding is unbound or unverified evidence and cannot discharge the required human-in-the-loop (HITL) obligation.

WebAuthn's normal assertion signs authenticator data and the hash of client data, not an arbitrary JSON object [8]. The transaction digest is therefore carried in the server challenge. The *relying party* is the service requesting authentication; the provider verifies its ID, the expected browser origin, challenge, user-presence and user-verification (UV) flags, credential ownership, and allowed algorithm. A hardware-bound profile additionally requires approved credential enrollment and *attestation*: evidence about the authenticator meeting the device policy. User verification alone does not prove a non-exportable hardware key; synchronized passkeys cannot silently satisfy that stronger profile.

5. A Component Map for a Changing Ecosystem

The component model supports a practical map of supplied roles and remaining integration work. A buyer can use it to locate a product in the architecture and identify the contracts still required. An implementer can use it to assemble a control plane from existing services and evaluate new tools as they appear.

Figure 3 classifies components by *responsibility* and *privilege placement*. An unassigned required role exposes an ownership gap; a supplied role with a missing interface, uncovered path, or weaker target guarantee exposes a contract gap. Gap discovery follows from the declared action's requirements. One product can supply several roles, and several products can jointly supply one role.

5.1. Nine Roles and Their Boundaries

Table 4 summarizes the nine roles. The Agent Plane hosts existing agentic workflows; the Control Plane owns admission and execution authority; the Enforcement Plane mediates protected effects. Evidence Providers contribute observations and judgments across these boundaries rather than forming another privileged execution plane. Evidence acquisition scheduling is an optional strategy within admission, not an additional mandatory component.

Placement	Logical Components	Boundary
Agent Plane	Agent Adapter	Frozen proposals and workflow correlation; no backing mutation privilege.
Control Plane	Policy Authority; Resource Registry; Admission Controller; Grant Authority; Outcome Service	Policy, observed context, admission, conserved authority, and reconciled outcomes. Observation and reservation ownership remain separate.
Enforcement Plane	Execution Gateway; Resource Adapter	Credential custody, grant redemption, exact API mapping, and declared target guarantees.
External contributors	Evidence Provider	Scoped candidate evidence; admission determines what it discharges.

Table 4: Component placement and authority boundaries. Full role contracts are defined in Section 4; implementations are assessed against these responsibilities.

5.2. Where Existing Tools Fit

Table 5 records selected capabilities documented by their maintainers, reviewed on 2 October 2026. It is an integration map, not a vendor conformance assessment. The final column states what an implementation must establish to satisfy the proposed KedgeFlow contract; it does not assert that the cited product cannot supply it. A local hook, an external authorization server, a resource transaction, and a signed receipt answer different questions and should be assessed separately.

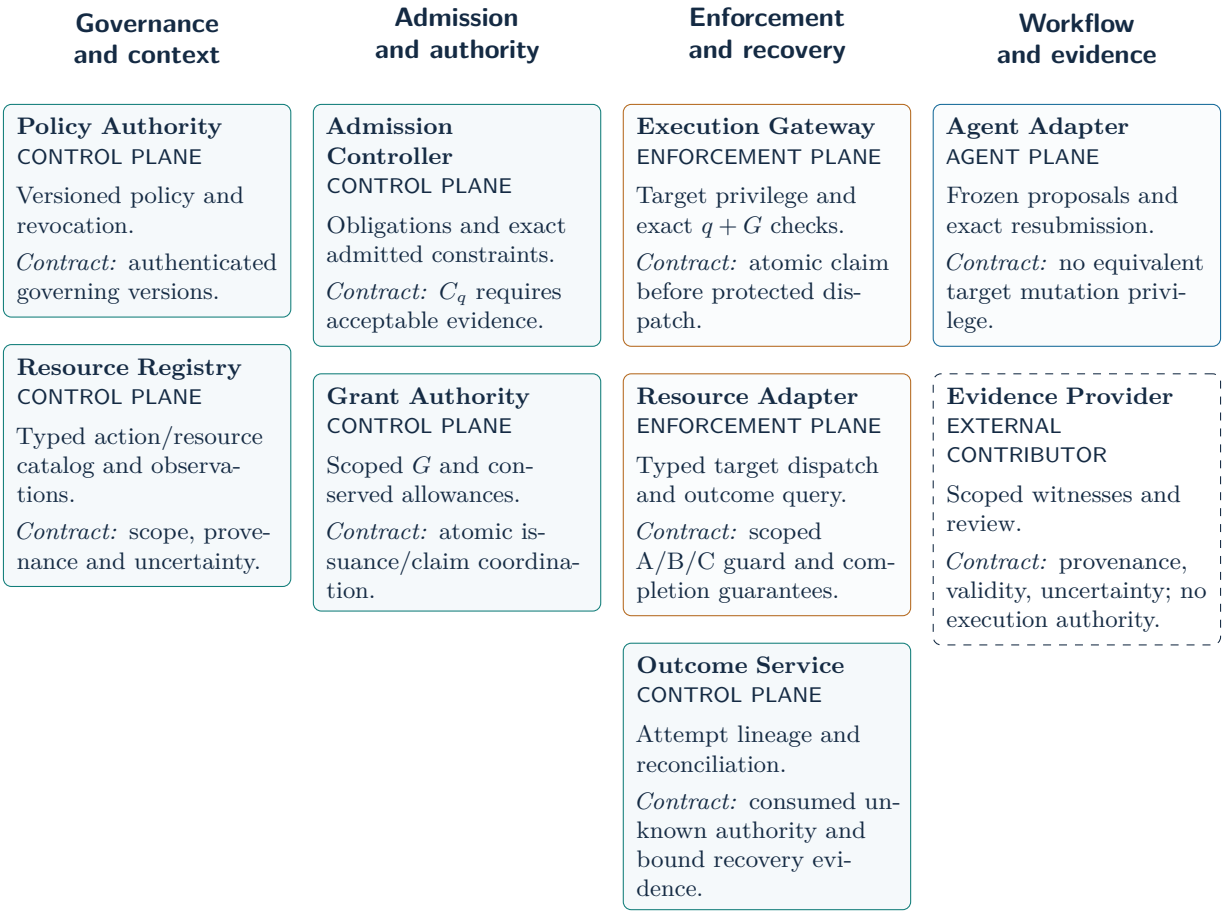


Figure 3: KedgeFlow Component and Contract Map. Columns group responsibilities; each card names its privilege placement and a required contract. The dashed Evidence Provider card is an external contributor, not a privileged control component. Figure 1 gives workflow wiring, Table 7 defines all six interfaces, and Table 5 maps candidate tools to the additional contracts they require.

Table 5: Documented tooling, component fit, and required integration contracts.

Tool or category	Documented function and role fit	Required KedgeFlow integration contract
OpenAI Agents SDK; Claude Agent SDK; Google Agent Development Kit (ADK)	Agent Plane runtimes with Agent Adapter attachment surfaces: function-tool guardrails, permission hooks/rules, and before-tool callbacks respectively [9, 10, 11].	Enumerate covered invocation paths, freeze the governed proposal before admission, preserve its identity across approval, and route protected effects through the Execution Gateway. Hook coverage varies by tool type and integration.
LangGraph	Orchestration runtime with persistence and human interrupts; graph nodes and tool wrappers can host the Agent Adapter [3, 12].	Bind resumed execution to the approved proposal and stable operation identity. Handle replay without repeating a protected effect; checkpoint state is distinct from resource commit evidence.
AutoGen; Microsoft Agent Framework	Single/multi-agent runtimes and orchestration. Agent Framework, the successor to AutoGen and Semantic Kernel, documents function middleware for validation and execution control [13, 14, 15].	Use middleware as an Agent Adapter surface, while documenting which tools it covers and which independent service owns credentials, grant verification, and effect mediation.
CrewAI	Agent crews and persistent Flows; local human-feedback and enterprise asynchronous review workflows [16, 17].	Bind review to the exact action rather than only a task description or generated output. Connect continuation to admission, grant redemption, and outcome retrieval.
MCP, revision 2026-07-28	Tool discovery and invocation transport; HTTP authorization establishes server access and audience validation [18, 19]. A governed reverse proxy can implement the Execution Gateway without replacing the agent runtime.	Negotiate the KedgeFlow profile, bind each consequential `tools/call` to a registered descriptor and frozen proposal, and require an atomic grant claim before upstream dispatch. `-32001 NEEDS_EVIDENCE` and `Mcp-KedgeFlow-Grant` are proposed extension contracts, not native MCP requirements.
A2A, version 1.0	Agent discovery, collaboration, task status, structured message elements called Parts, and capability extensions [20]. Its Agent Plane messages carry references to delegated execution roots.	Advertise the required assurance profile in an AgentCard extension; carry a typed root-reference Part; authenticate retrieval and delegate child grants under the same root generation. A2A transports the reference; the Grant Authority owns allocation and conservation. Task completion is not resource commit evidence.
Open Policy Agent (OPA)	Policy evaluation within the Admission Controller. OPA separates decision from enforcement and supports signed policy bundles [21, 22].	Assign a Policy Authority, record bundle identity and input provenance, define assurance obligations, and connect the decision to enforced issuance and redemption. Policy evaluation remains a reusable part of the chain.
Cedar	Deterministic principal/action/resource/context evaluation for the Admission Controller; schema-based policy validation, default deny, and forbid-overrides semantics [23, 24].	Validate the policy/schema pair and actual request/entities; bind verified witness facts to inputs. Since Cedar skips individual policies that error, the KedgeFlow wrapper must inspect diagnostics and fail closed on evaluation errors. `Allow` alone does not establish that external evidence obligations are fulfilled.

Tool or category	Documented function and role fit	Required KedgeFlow integration contract
OpenID Connect (OIDC) providers; SPIFFE and its SPIRE implementation; OAuth credential services	Subject authentication and workload authentication are distinct inputs to the Agent Adapter, Admission Controller, and Execution Gateway [25, 26, 27]. DPoP and token exchange provide token-binding and delegation building blocks [28, 29].	Verify the subject/workload pair and delegation scope; retain target credential custody in the Enforcement Plane. DPoP is not proposal-body signing or proof of business authority. An upstream credential service must implement the selected exchange/federation binding and preserve the grant's scope.
etcd 3.6; transactional stores	Substrates for Resource Registry observations and the Grant Authority's ledger. etcd atomically evaluates comparisons and executes one transaction branch [30, 31].	Compare the root generation, nonce/claim state, and required reservations in one authoritative transaction. Define conflict domains, failure recovery, and fencing. Key prefixes organize records but do not create independent Raft groups; a ledger transaction does not atomically change an enterprise resource.
Temporal	Durable workflows and Activities; a substrate for evidence acquisition, approval, Outcome Service reconciliation, and orchestration of saga compensations [32, 33, 34].	Bind retries to stable operation identities and reconcile unknown effects before re-execution or compensation. Fund separately admitted compensations under the conserved saga root. Durable orchestration does not ensure that compensation is possible, succeeds, or restores an exact prior state.
Envoy; API gateways	Request mediation. Envoy's external-authorization filter calls a separate decision service [35].	Configure and test complete coverage and fail-closed behavior; add action-bound grant verification, shared redemption state, protected credential custody, and outcome handling to implement an Execution Gateway.
Kubernetes; conditional resource APIs	Resource Adapter substrate. Kubernetes conditional updates use `resourceVersion` to reject stale writes [36].	Advertise actual guard scope and strength, bind the native condition to admission assumptions, and report conflict or uncertainty. A per-object condition is not a transaction over every cross-resource invariant.
OpenTelemetry; monitoring, validators, semantic judges	Evidence Provider inputs: telemetry, deterministic checks, and judgments. OpenTelemetry standardizes generation, collection, and export of traces, metrics, and logs [37].	Wrap results as witnesses with proposal/resource binding, producer identity, observation time, validity interval, coverage, and uncertainty. A log entry or judge score is not automatically trusted evidence or authority.
Human review; WebAuthn	An Evidence Provider can bind review to a server-issued challenge incorporating the exact proposal digest and policy epoch. WebAuthn authenticates an assertion using registered public-key credentials [8].	Verify the assertion, one-time challenge, origin, relying-party identifier, user presence/verification, reviewer authority, and template digest. A claim of hardware-bound approval additionally requires a credential enrollment and attestation policy; WebAuthn does not directly sign arbitrary JSON.
Sandboxes, e.g. gVisor	Isolation for Agent Plane processes or constrained execution. gVisor isolates applications from the host through a userspace application kernel [38].	Declare filesystem, network, and credential access. Isolation supports containment; the deployment must still establish the protected enterprise action's admission and enforcement contracts.

Existing controllers and governance proposals. KedgeFlow also builds on work that directly addresses control. Microsoft's Agent Control Specification (ACS) specifies deterministic, fail-closed intervention-point verdicts and host obligations, including approval identity binding [39]. Agent Infrastructure Control Protocol (AICP) proposes a lifecycle linking freshness-labelled situations, exact plans, approval, execution, verified outcomes, and recovery [40]. Proof-Carrying Agent Actions (PCAA) and Canonical Action Verification and Attestation (CAVA) describe portable action certificates, canonical action identity, approval binding, and receipts [41, 42]; Fernandez's Agent Control Protocol (ACP) addresses stateful temporal admission [43]. These drafts and working papers overlap individual KedgeFlow roles. Their publication does not establish conformance to the complete architecture, and this whitepaper does not report independent validation of their implementations. KedgeFlow's proposed contribution is the explicit composition of responsibilities and interfaces, not a claim to invent every underlying mechanism.

5.3. Governed MCP Proxy Binding

The *Governed Reverse MCP Execution Gateway* is a deployment pattern for the Execution Gateway role, not a tenth component. An existing harness keeps its model, planner, graph, and ordinary tool integration. Its MCP endpoint resolves to a protected gateway; an Agent Adapter supplies the admission workflow and grant carriage. The gateway terminates client authentication, resolves tool identity against a trusted Resource Registry, and opens a separately authenticated upstream connection. The upstream mutation credentials are inaccessible to the Agent Plane, and direct upstream access to governed effects is blocked by the declared deployment boundary.

The binding pins MCP revision 2026-07-28 and the proposed `KF-MCP-0.1` profile. MCP request metadata includes the required protocol version and client capabilities; client identity metadata is self-reported and is not workload authentication [44]. Modern MCP negotiates extensions per request rather than through an initialization handshake [45]. The client declares the `io.kedgeflow/governed-mcp` extension in its capabilities' `extensions` map with `profile: KF-MCP-0.1`; the gateway advertises the same supported profile through discovery.

The proposed profile adds `-32001 NEEDS_EVIDENCE`, a structured `RemediationContract`, and the HTTP `Mcp-KedgeFlow-Grant` header. A RemediationContract names missing evidence and the supported admission and retry path; it supplies no execution authority. A signed *envelope* packages a typed payload with its schema, issuer-key reference, and signature metadata. The header contains unpadded base64url encoding of the complete signed `KF-JSON-0.2` grant envelope. These additions require documented discovery/configuration and explicit client support. A runtime without that support cannot be assumed to parse a custom error, obtain evidence, or attach a grant merely because it supports MCP.

Unsupported protected invocations fail closed; a deployed compatibility adapter may expose a standard MCP tool error with `isError=true`, but it must preserve the same proposal identity and admission semantics. Native MCP distinguishes protocol errors from recoverable tool execution errors [18]; the custom admission response is a profile decision, not a change to that distinction.

The governed invocation lifecycle is:

1. The gateway authenticates the caller and classifies the registered tool using its versioned Consequential Tool Descriptor (CTD). Unknown tools, changed input schemas, and undeclared invocation paths are rejected. Server-supplied annotations are not accepted as authority to downgrade consequences. A `ReadOnly` classification still requires access authorization: a read may disclose protected information, trigger a billable operation, or return hostile content. An authorized epistemic call may pass through with structured, redacted telemetry.

2. For a consequential `tools/call` without a redeemable grant, the gateway binds the registered tool identity and canonical arguments and returns a non-dispatching remediation response. The Agent Adapter freezes full proposal q at admission; a pre-proposal response marks its proposal digest as null. A response for an existing proposal binds $H(q)$. The contract supplies unmet obligations, profile/carriage, an expiry, and an authenticated admission endpoint. The admission flow establishes the active policy epoch for the exact proposal. The error does not carry target credentials or authorizing authority. The adapter validates the endpoint against its configured trust domain rather than treating an arbitrary tool error URL as trusted.
3. The Agent Adapter gathers witnesses and invokes admission for that exact proposal. A valid certificate C_q records discharged obligations; the Grant Authority issues G under the root ledger. Changing arguments, target incarnation, schema, or a required policy binding requires fresh admission. The adapter resubmits the same logical operation with a new JSON-RPC request identifier and the profile's grant carriage. MCP request identifiers correlate transport exchanges; they are not execution idempotency keys.
4. The gateway verifies the proposal and grant bindings, subject/workload identity, proof-of-possession, expiry, policy epoch, and root generation. It atomically claims the allowance before opening target execution. It then obtains a narrowly scoped target credential through the configured exchange or federation service, invokes the Resource Adapter's declared guard, and records the resulting operation identity and outcome evidence. The header is consumed at the gateway, excluded from model-visible telemetry, and removed before upstream tool dispatch.
5. The Outcome Service distinguishes rejected admission, rejected guard, confirmed effect, and unresolved effect. A transport timeout after a possible dispatch yields `OUTCOME_UNKNOWN`; the adapter retrieves the operation outcome instead of minting a new root or blindly retrying the mutation.

The MCP-facing authorization token is audience-bound to the governed endpoint. MCP authorization prohibits accepting or passing through unrelated target tokens [19]. KedgeFlow therefore separates MCP server access from the target credential minted after a successful claim. The exchange service's policy maps the verified user delegation, workload identity, and G to a target-specific credential. RFC 8693 specifies the OAuth token-exchange request; it does not make every AWS session, database token, or software-as-a-service (SaaS) credential a native token-exchange result. Each target needs an explicit supported binding, downscoping rule, lifetime, and enforcement contract [29, 46]. Credential issuance and resource mutation are separate failure points, and neither enlarges the already claimed allowance.

5.4. A2A Delegation Binding

A2A 1.0 supplies agent discovery and task transport; KedgeFlow supplies conserved execution authority. The proposed `urn:kedgeflow:a2a:root-reference:1.0` extension declares assurance requirements through `AgentCard.capabilities.extensions` and is activated using the binding's extension mechanism. For HTTP, the client sends `A2A-Version: 1.0` and the extension URI in `A2A-Extensions`. The public card at `/.well-known/agent-card.json` advertises the extension as required for governed skills; its `params.assuranceProfiles` lists the accepted profile identifiers. These are proposed KedgeFlow extension fields, not additions to the core A2A data model [20].

The following is a complete structured Part example. A2A 1.0 uses the presence of `data` as the Part discriminator; it does not use the legacy `kind: data` field. The type URI below identifies the KedgeFlow payload schema, while `mediaType` identifies its serialization.

Here a *Part* is an A2A message element carrying structured data. The root reference identifies ledger state to resolve through an authenticated authority endpoint; it does not authorize the receiving agent to execute.

```
{
  "mediaType": "application/json",
  "data": {
    "type": "urn:kedgeflow:root-reference:1.0",
    "root_id": "saga-drain-20261002-41",
    "root_generation": 7,
    "delegation_id": "delegation-drain-check-1",
    "assurance_profile": "cluster-maintenance-v1",
    "authority_endpoint": "https://control.example.org/kedgeflow/v1"
  }
}
```

The Part contains a reference to authenticated ledger data, not a bearer grant, delegation token, or claim of control-plane ownership. The receiver's Agent Adapter authenticates to the configured authority endpoint, resolves the delegation descriptor, checks the intended workload/subject pair and the root generation, and obtains a child certificate for the precise delegated action. The Grant Authority performs `delegate(gid, childCertificate)` under the parent generation; it transfers or reserves child allowance in the same conserved ledger boundary before a child grant can be claimed. A task message cannot create a new allowance, enlarge the resource/action set, advance the root generation, or authorize further delegation beyond the parent's permissions.

This binding permits a planner, specialist, validator, or infrastructure agent to remain in its existing A2A workflow. The public AgentCard communicates what evidence and adapter guarantees the delegated skill requires; authenticated ledger resolution determines whether a particular child task has that authority. Capability advertising is not attestation of successful enforcement. A2A task and context identifiers are recorded alongside the execution root and operation identifier, but do not replace them. Every protected child effect follows the same Admission, Execution, and Outcome Interfaces; a completed A2A task is reconciled against the target receipts before its mutation is reported as confirmed.

5.5. Policy and Saga Substrates

A Cedar integration treats policy evaluation as one deterministic stage of admission. The Policy Authority publishes a versioned policy/schema pair and separately defines the assurance obligations. The Admission Controller validates the request, entity graph, context types, and verified witness bindings before invoking Cedar; it records the input digest, determining policies, decision, and diagnostics. Cedar's `Allow` or `Deny` result does not natively collect witnesses or establish their provenance. An absent witness is represented as unmet evidence, not as an agent-supplied context boolean set to true. Because Cedar excludes a policy whose evaluation errors, the KedgeFlow admission wrapper rejects such diagnostics rather than allowing a failed safeguard to disappear from the admission decision [23, 24].

Temporal can persist the Outcome Service's reconciliation and compensation orchestration, while the Resource Registry describes each registered compensating action and the Grant Authority conserves its budget. This separation extends the original saga decomposition of a long-running transaction into local transactions with compensating transactions [47]. A compensation is a new governed effect with its own proposal, state guard, deadline, and receipt. It may be unavailable, only partially restorative, or invalid after another writer changes state. The service therefore registers compensation intent before the forward

dispatch, reserves recovery allowance under the saga root, and runs only the compensations justified by known outcomes and their current guards [34]. An unresolved timeout first enters reconciliation; it cannot justify blind retry or a presumed inverse. Compensation failure or insufficient reserved authority is an explicit recovery outcome, not a declaration that the workflow has rolled back.

5.6. A Deployment Gap Register

A gap is assessed against a *named deployment, protected resource scope, threat model, and required guarantee*. The following status terms describe the implementation of each role, rather than the maturity of the whole industry:

- **Absent (missing):** no implementation is assigned to a required responsibility.
- **Implicit:** behavior exists inside a hook, script, workflow, or service, but its owner, boundary, or callable contract is unspecified.
- **Partial (incomplete):** an implementation covers only part of the required scope or lifecycle, or supplies weaker guarantees than the deployment requires.
- **Unverified:** the capability is claimed, but documentation or test evidence does not establish the required behavior. Unverified is an evidence status and can accompany any claimed implementation; it is not evidence of absence.

Table 6 turns these distinctions into an observable gap inventory. A deployment records the applicable status, responsible implementation, protected scope, and test result for every row. The examples are failure patterns to investigate, not findings about every tool in Table 5.

Table 6: Deployment gaps tied to components and observable acceptance tests.

Component	Example gap and status	Observable acceptance test
Agent Adapter	Implicit: approval covers a description, while resumed parameters can change. Partial: some tool paths bypass the adapter.	Change a protected parameter after approval and invoke each declared tool path. Execution must require a new admission for the changed action; unsupported paths must fail closed.
Policy Authority	Absent: no policy owner or version service. Partial: revocation exists without a bounded freshness contract.	Revoke authority and partition a client. After the declared synchronization bound, new dispatches must stop; decisions must identify the governing policy version.
Resource Registry	Implicit: resource facts live only in agent context. Partial: inventory lacks versioned CTDs, target incarnation, covered writers, or actual guard/compensation contracts.	Recreate a target, change a tool schema, or age an observation. Resolution must expose the changed identity/digest or unknown state; admission must reject stale bindings and unavailable recovery assumptions.
Admission Controller	Implicit: a judge score directly drives execution. Partial: policy permits are accepted despite missing evidence or evaluation diagnostics.	Remove/expire a witness, substitute another subject/workload pair, or trigger a Cedar evaluation error. Deny or defer with the unmet obligation; neither a recommendation nor `Allow` alone discharges it.

Component	Example gap and status	Observable acceptance test
Grant Authority	Absent: an allow verdict yields no independent grant. Partial: grants lack exact scope, shared reservations, or conserved root/saga generation semantics.	Race claim against delegation or duplicate child allocation in etcd. Reject changed bindings and conserve forward/recovery allowance; retain reservations for unresolved effects.
Execution Gateway	Partial: tool hooks leave ambient target credentials or direct API access. Unverified: replicas claim single-use redemption without a race test.	Attempt bypass; change MCP arguments; redeem one grant through two replicas. Bypass/mismatch must fail and at most one dispatch authorization may succeed; target credentials must remain in the Enforcement Plane.
Resource Adapter	Implicit: a pre-action read is assumed to remain valid. Partial: revalidation is presented as an atomic commit guard.	Change relevant resource state between admission and execution. The adapter must reject under its declared guard or report the weaker guarantee and uncertainty explicitly.
Outcome Service	Absent: no outcome lifecycle. Partial: dispatch success or a failed directed acyclic graph (DAG) workflow is treated as confirmed completion or rollback.	Drop a possible mutation's response and fail a later child. Preserve `OUTCOME_UNKNOWN`, reconcile the effect, and admit only justified compensations; report failed or partial recovery explicitly.
Evidence Provider	Implicit: logs or human messages serve as unbound evidence. Partial: WebAuthn verifies a login but approval is not bound to the reviewed action.	Replay/retarget a witness; replace the template or proposal after review; use an unenrolled credential. Reject it. A hardware assurance profile must also reject missing required credential-attestation evidence.

Passing these checks establishes evidence for the tested scope and assumptions, not a universal safety guarantee. In particular, authority-ledger reservations cannot protect against actors outside the mediation boundary, and a gateway-signed receipt attests the gateway's report rather than necessarily a durable resource commit. The Resource Adapter and Outcome Service must distinguish a dispatch attempt, a confirmed result, and an uncertain effect using evidence the target actually supplies.

5.7. Classifying the Next Tool

Classify a new tool against a named action surface, not its marketing category:

1. Identify the supplied obligation, callable inputs/results, owned records and keys, and actual permission to affect the target.
2. Assign only those functions to the full component names. Record wrappers, trust enrollment, target prerequisites and interfaces still required.
3. Apply the gap statuses in Section 5.6, and attach documentation or an executed test to every claimed contract. A passed case establishes verified evidence for that case's scope and assumptions; it is not a permanent product-wide status.

For example, a hypothetical new policy checker returning a permit decision is an Admission Controller substrate. It is partial for that role until acceptable evidence, exact proposal/constraint binding and certificate issuance are implemented. If policy activation and revocation are handled only by an unowned

script, the Policy Authority contract is implicit; if no grant issuer is assigned, the Grant Authority role is absent. Tests for the checker do not verify gateway custody or target guards. This procedure identifies required integration work without predicting a new class of product.

A deployment is complete when every required obligation has an explicit owner and verified contract evidence across the declared protected scope, irrespective of how many products supply its roles.

6. Programmable Interface Contracts

The component map identifies who owns each responsibility. The six interfaces define how those owners exchange proposals, authority, evidence, and outcomes. Agent frameworks, controllers, evidence tools, and enforcement systems can then evolve independently while preserving the contract. This revision supplies semantic contracts and a candidate wire profile: concrete transport, encoding, and signature rules. Independent peer exchange and target-specific production qualification remain separate evidence requirements.

OpenFlow standardized the controller–switch interface, including commands to modify forwarding behavior and messages reporting datapath events [4]. The corresponding boundary here is the **Enforcement Control Interface** between the Control Plane and an Execution Gateway. Agent proposals, governance administration, and evidence acquisition need separate contracts: combining them into a single "southbound API" would obscure which participant exercises authority. Unlike a switch instruction, an agent action can depend on uncertain evidence, changing resource state, and asynchronous physical effects.

6.1. Six Interfaces and Their Participants

Table 7 identifies the caller, callee, request, and result for each interface. A product can implement several components in one process, but it must preserve these logical responsibilities and authorization checks.

Interface	Caller → Callee	Request	Result and Obligation
Governance Interface	Authorized operator → Policy Authority or Resource Registry	`publish`: policy, scope, expected epoch; `revoke`: grant/root/scope, reason; `register`: descriptor and expected version.	Authenticated versioned policy, revocation, or descriptor record. Administrative writes require scope permission and reject conflicting versions.
Runtime Admission Interface	Agent Adapter → Resource Registry, Admission Controller, Grant Authority	`discover`: scope; `propose`: exact q, evidence references; `grant`: C_q, audience, requested lifetime/completion mode.	Typed resource/action descriptors; `Admitted`(C_q), `Denied`, or `NeedsEvidence`; then an exact grant or refusal. Discovery and admission confer no execution privilege.
Enforcement Control Interface	Authorized Control Plane component → Execution Gateway	`install`: signed policy/epoch, issuer keys, grant records, revocations, expected gateway version.	Acknowledgment of installed version and coverage, or rejection. Installation cannot broaden the signed authority; stale gateways stop accepting affected actions.

Interface	Caller → Callee	Request	Result and Obligation (continued)
Execution Interface	Agent Adapter → Execution Gateway	`execute`: exact q, G, dual-identity proof, request identifier; `status`: attempt identifier.	Signed attempt receipt or status: rejected before dispatch, dispatched, pending, committed, failed, or unknown. Repeated status queries do not dispatch again.
Evidence Interface	Admission Controller → Evidence Provider; authorized provider/gateway → Resource Registry	`evaluate`: obligation, $H(q)$, state reference, policy epoch, deadline; `observe`: scoped observation with producer identity and time.	Candidate evidence or stored observation reference, with provenance, version, validity, and limitations. The registry checks producer scope; admission decides what the record discharges.
Outcome Interface	Execution Gateway → Outcome Service; Outcome Service → Gateway for status	`report`: attempt receipt, target operation reference, available post-state evidence; `status`: unresolved attempt.	Durable acknowledgment, linked trace, or unresolved reconciliation ticket. Authoritative target queries can refine status; lack of evidence remains unknown.

Table 7: The six proposed KedgeFlow interfaces. Operations name semantic responsibilities, not mandatory URL paths.

Registry registration uses the Governance Interface, and observation ingestion uses the Evidence Interface. Observation records bind producer identity and authorized scope, resource incarnation, schema, value/digest, source and observation times, validity, and uncertainty. An observation describes what was seen, not a reservation of future capacity. An agent may contribute observations only with explicit producer permission; its own report does not become trusted target evidence. Within the Enforcement Plane, an Execution Gateway invokes a Resource Adapter through `describe`, `dispatch(verifiedAction, guard, attemptId)`, and `query(attemptId)`. The adapter declares its action schema, covered resource incarnation, state predicate, conditional-write support, idempotency behavior, and completion modes. Raw agent parameters cannot enter this privileged dispatch path without gateway validation.

6.2. Shared Records: Exact Scope Before Transport

Implementers must agree on the meaning of records before selecting HTTP, RPC, or another binding. Canonical resource identity includes an *incarnation*: deleting and recreating a resource at the same name does not preserve an old grant. Each action has a versioned schema; absent fields, units, defaults, and parameter normalization must have unambiguous meanings.

Proposal q. Proposal identifier, authenticated principal, intent/context, canonical action and schema version, resource identifiers/incarnations, exact parameters, dependencies, and referenced observed state. The intent text explains the request; it cannot override the typed action or scope.

Admission Certificate C_q. Exact proposal digest, policy epoch/scope, discharged obligations, evidence manifest with validity requirements, admission time, certificate expiry, and the digest of approved execution constraints κ. These constraints include the guard/predicate scope, adapter descriptor and required level, allowed audience and completion mode, maximum lifetime, ownership epoch,

and evidence freshness at the required check points. The Grant Authority must not broaden them. `NeedsEvidence` returns unresolved obligations and *no* certificate; it is not conditional admission.

Execution Grant G**.** Proposal/certificate digests, principal, exact action/resources/parameters, audience, execution root and generation, nonce, reservation/fencing references, state guard, policy epoch, validity interval, and completion mode. A gateway rejects any mismatch rather than repairing the action.

Evidence record. Question/obligation and proposal digest, referenced resource incarnation/state, result including unknown or contradiction, provider identity/method version, provenance and exposure domain, observed time, validity interval, and scope limitations. The exposure domain records shared sources or failure dependencies relevant to judging provider independence. A signature authenticates the provider's statement, not its truth.

Attempt receipt. Attempt, proposal, grant/root/generation/nonce identifiers; gateway and target identities; dispatch/completion mode; guarantee level; policy and guard checks; target operation reference; timestamps; outcome and supporting observations. A receipt distinguishes accepted dispatch from confirmed commit.

6.3. Obligations on Agents and the Control Plane

An agent integration implements the Agent Adapter contract: freeze each tool call into an exact proposal, authenticate with workload identity, preserve proposal/grant/attempt correlation, and deliver denied, missing-evidence, pending, and unknown results faithfully to the workflow. It submits mutations through the Execution Interface and uses status queries after ambiguous replies. It must not interpret a timeout or denial as permission to call the underlying tool directly. Removing ambient mutation privilege is a deployment obligation; a client shim alone cannot ensure it.

The Control Plane must publish versioned policy and revocation semantics, maintain typed resource/coverage descriptors, evaluate evidence under explicit freshness and trust rules, and issue grants only after successful admission. The Grant Authority and gateways must share an atomic authority ledger for issuance, replacement, delegation, reservation, and claim (Section 7). Enforcement installation returns a version acknowledgment; an operator cannot assume that publishing a revocation immediately cancels an already dispatched target operation. The Outcome Service retains attempt lineage and reconciles uncertainty without inventing completion or granting a retry.

6.4. A Proposed Cryptographic Binding

`KF-JSON-0.2` is the proposed v0.9 binding using mutual Transport Layer Security (mTLS) over HTTPS, the RFC 8785 JSON Canonicalization Scheme (JCS) [48], SHA-256 digests [49], and Ed25519 signatures [50]. JCS gives a conforming JSON value one canonical byte representation. *Domain separation* uses purpose-specific byte prefixes so a signature or digest is bound to its intended use. Section 6.9 fixes the envelope and exact signing bytes.

Payload definitions bind the applicable operation, identities, policy, scope and validity; each record kind carries its required fields. Deployments additionally specify key enrollment/rotation, canonicalization rejection, replay handling, and clock-uncertainty policy. Naming these primitives does not supply a complete interoperable protocol or justify an arbitrary universal clock bound.

A signed gateway receipt authenticates what that gateway reported. It is not automatically a target-signed proof, an immutable audit record, or a guarantee of legal non-repudiation. Tamper-evident

retention requires a separately specified storage, key-custody, access-control, and verification design; target operation references and observations strengthen an outcome claim only to the extent the target supports them.

6.5. Failure Results and Recovery

Failures preserve the boundary. `IDENTITY_CONFLICT`, `GRANT_MISMATCH`, `POLICY_STALE`, and `EVIDENCE_EXPIRED` prohibit dispatch. `NEEDS_EVIDENCE` requests additional evidence without admitting the proposal; `DENIED` records a policy refusal. `ROOT_UNAVAILABLE` or `NONCE_CONSUMED` requires inspection of the existing root/attempt rather than new authority. `STATE_GUARD_VIOLATION` means the declared predicate failed. `OUTCOME_UNKNOWN` creates a reconciliation ticket and retains the claim and any reservation. No failure result permits unmediated execution.

6.6. Dual-Identity Execution Context

The principal combines the subject/workload pair with proof-key and delegation bindings:

$$p = \langle u, w, j, d_{\text{del}} \rangle, \qquad u = \langle \text{iss}, \text{sub} \rangle, \quad w = \text{SPIFFE ID}, \quad j = \text{DPoP key thumbprint}. \tag{1}$$

u combines the trusted issuer iss and its subject identifier sub; w identifies the requesting workload; j is the key thumbprint, a digest identifying the access token's proof key; and d_{del} is the digest of the validated delegation context. OIDC subject identifiers are scoped to their issuer. A display name, email, workload identity, or ID token alone is not an OAuth authorization to perform the target action [25, 26, 28].

The Agent Adapter **MUST** authenticate its workload with a valid SPIFFE SVID and present an authorized user delegation access token with a DPoP proof. The gateway verifies the token issuer, audience, lifetime, scopes and confirmation-key binding; proof method/URI, issuance time, nonce/replay rules, and access-token hash; and the local policy allowing this workload to act for this subject. RFC 9449 proves possession of the bound key. It does not itself establish the delegation, the human's transaction approval, or request-body integrity. The frozen proposal and grant supply the exact body binding.

The proposal, certificate, and grant **MUST** preserve u, w, j and d_{del}, either in explicit fields or through the validated exact-proposal and delegation digest chain. The certificate's proposal digest transitively binds its DPoP key. The authenticated execution caller must match them. A new subject token or rotated proof key requires policy-defined continuity and revalidation, not identity substitution. No user access token or workload SVID is treated as ambient backing mutation privilege.

6.7. Gateway-Side Credential Minting

After validating G and atomically claiming its allowance, the Execution Gateway may request a short-lived target access token through an authorized RFC 8693 token-exchange service, or *credential broker* [29]. The broker authenticates the gateway as a client, validates subject and actor tokens, and intersects user delegation, workload permission, grant scope, CTD scope and target policy. A target credential cannot widen G.

Exchange Field	Source	Constraint
`grant_type`	RFC 8693 token-exchange grant URI.	Fixed to `urn:ietf:params:oauth:grant-type:token-exchange`.

Exchange Field	Source	Constraint
`subject_token`, `subject_token_type`	Validated delegated user access token and declared token type.	Issuer/audience and delegation verified; ID tokens are not substituted. Tokens are not logged or sent to arbitrary upstreams.
`actor_token`, `actor_token_type`	Workload or gateway actor credential accepted by the exchange service.	A SPIFFE JSON Web Token (JWT) SVID is usable only under an explicit issuer/audience trust agreement. Preserve human subject and workload actor semantics.
`audience` / `resource`	Registered target token audience/resource.	Derived from CTD/grant, not a caller-selected URL. The gateway audience and target-token audience are distinct.
`scope`	Intersection of requested action and broker policy.	Bound to the least target privilege available; finer exact-action/fence constraints remain enforced by gateway/adapter.
Exchange result	`access_token`, `issued_token_type`, `token_type`, `expires_in`, effective `scope`.	Validate target, type, scope and remaining lifetime. Keep token only in the enforcement boundary; reject broader or unusable credentials.

Table 8: RFC 8693 credential-minting contract. KedgeFlow grant digests and action constraints are broker policy inputs under a negotiated extension, not standard RFC 8693 fields.

A target authorization server must support the chosen exchange and actor semantics. AWS Security Token Service (STS) sessions, database authentication tokens and SaaS OAuth tokens use their supported provider-specific adapters; RFC 8693 does not make every target implement a common minting API [46]. If the credential cannot express an exact action, the independent gateway still checks exact action binding against $H(q)$, fencing, guards and authority conservation. Token acquisition failure leaves a claimed attempt durably undispatched; retry or retirement follows the no-effect/fencing contract, not a new independent grant.

Budget the postclaim exchange. Token acquisition occurs after the atomic claim and before target dispatch. Measure connection setup, provider queueing, exchange/body time and the final ledger check; reuse authenticated connection pools rather than opening a new connection per call. A latency spike of 500 ms is an illustrative test case, not a universal identity-provider (IdP) or STS threshold.

The Grant Authority chooses a lifetime within certificate, evidence, reservation and policy bounds; the gateway's exchange deadline leaves an adopter-set margin for the final authority check and dispatch. If that budget is unavailable, refuse rather than silently extend the signed expiry t_{expire}.

The gateway **MUST** recheck grant and credential validity after exchange and immediately before dispatch, including time spent on the final ledger round trip. A local `expires_in` estimate must conservatively account for response transit time [51]; clipping the local timestamp cannot make an actually broader bearer credential narrower. Timeouts retain the claim and a durable undispatched record when no request was sent. Once a target request may have been sent, cancellation or grant expiry is not proof of no effect. Dispatch-bound expiry governs initiation; commit-bound expiry additionally requires enforcement at the target commit point. Pooling improves connection reuse, not authority validity.

The gateway **MUST NOT** forward an MCP client authorization token to an upstream target. It strips incoming grant, authorization, DPoP and identity-spoofing headers before constructing a target request with the broker-issued credential. Protected credentials and exchange client secrets are absent from the Agent Plane and evidence providers.

6.8. The MCP Remediation Contract

The Governed MCP Proxy profile carries one signed `KF-JSON-0.2` grant envelope as base64url canonical JSON in the negotiated `Mcp-KedgeFlow-Grant` header. The header authorizes only the exact bound `tools/call`, target incarnation, subject/workload and audience. Protected batches are rejected; MCP session identifiers and reused HTTP headers never create new authority.

Header budgets and optional references. The encoded envelope grows by roughly one third. Its field and the aggregate request headers must fit the smallest intermediary budgets, including authorization, DPoP and proxy-added fields. NGINX defaults to an 8K buffer per header field; AWS Application Load Balancer (ALB) caps one field at 16 K and total request headers at 64 K. Cloudflare's content delivery network (CDN) increased its header support to 128 KB in October 2025, so 8/16 KB is not a common limit across these products [52, 53, 54]. Actual configurations and the complete route remain deployment checks.

The optional proposed `KF-GRANT-REF-0.1` transport is separately advertised and activated by client and gateway. Exactly one `Mcp-KedgeFlow-Grant-Ref` value carries the lowercase 64-hex digest of the *complete signed envelope*, using the existing domain-separated grant digest from Section 6.9. It is an opaque immutable reference, not a caller-selected URL or new authority. The gateway authenticates the caller and tool scope before a bounded lookup at an operator-pinned, authenticated resolver. Retrieval authorization covers the gateway and requested subject/workload/delegation scope. Redirects, nested references, duplicate/mixed carriage and oversized responses are rejected. Caller access tokens, DPoP proofs and target credentials are not forwarded to the resolver.

After retrieval, the gateway verifies the original schema, signature, digest, identity, action, audience and validity, then uses the same atomic ledger claim and final policy/generation checks as inline carriage. Lookup failure confers no allowance and cannot trigger fallback dispatch. Cached bytes never bypass current validation; changing carriage does not reset a nonce. Reference usability ends with grant validity, and deleting a stored envelope cannot restore consumption.

The signed KF-JSON-0.2 envelope and RemediationContract remain unchanged: reference support is an additional transport capability, not a replacement `grant_carriage` field. The reference slice uses a 6 KiB inline value cap, a 64 KiB resolved-envelope cap and a two-second lookup cap; these are testable profile choices, not universal network limits. The distributed `spec/GRANT_REFERENCE_PROFILE.md` fixes capability activation, the lookup request and refusal/retry rules.

The supplied implementation is a resolver client and gateway binding, not a production store. The Grant Authority owns authorized publication and record custody; gateway/service operators enforce retrieval policy and protect the network path. Deployments must also bound lookup rate, storage access and aggregate HTTP headers.

For a consequential call without authority, the KedgeFlow extension returns the JSON-RPC server error code −32001 with a `RemediationContract`. It contains an exact proposal digest when available (otherwise explicitly null), the versioned action/descriptor reference, canonical parameter digest, remediation identifier, obligations with acceptable evidence classes, admission endpoint, profile/carriage, expiry and retry conditions. The authenticated admission response and certificate establish the active policy epoch for the frozen proposal. `NEEDS_EVIDENCE` is a missing-authority negotiation state, not conditional admission. This custom error and header are not native MCP guarantees; clients and servers must advertise and activate the extension. The Agent Adapter recognizes remediation only when the negotiated profile, `NEEDS_EVIDENCE` message and schema-valid contract all agree. A bare `-32001` or

local timeout is insufficient and cannot initiate replacement authority. Ordinary tool execution errors retain MCP semantics.

The Agent Adapter requests or supplies missing witnesses, obtains C_q and G, and resubmits the same canonical action with fresh caller proofs. A changed action starts a new admission. Denial, expiry, an unreachable controller, or an unsupported extension cannot fall back to direct mutation. The complete typed remediation fixture and schema are included with the reference artifacts.

6.9. KF-JSON-0.2 Encoding and Signature Contract

The accompanying schema uses JSON Schema Draft 2020-12 and defines *closed objects*: unknown fields are rejected unless a field explicitly supplies an extension container. Each signed *envelope* wraps a typed payload with `profile`, `kind`, `schema`, `key_id`, and `signature`. The profile is `KF-JSON-0.2`; the schema identifier is the proposed versioned identifier in the distribution, not evidence that a public registry or standards body has ratified it. Implementations resolve this identifier against a pinned local schema, rather than trusting network retrieval.

Signature bytes are UTF-8 `KedgeFlow/KF-JSON-0.2`, one zero byte, and RFC 8785 JCS of the envelope without `signature`. Ed25519 signatures use unpadded base64url; key IDs resolve to independently trusted, role-authorized public keys. The proposal digest is SHA-256 of UTF-8 `KedgeFlow/KF-JSON-0.2/proposal`, one zero byte, and JCS of `{schema,payload}`. Other signed-record digests append `/<kind>` to the profile prefix and cover JCS of the entire signed envelope. Parameter, delegation, and constraint digests use the corresponding `/parameters`, `/delegation`, and `/constraints` domain and canonical field value. All digests are lowercase 64-character hexadecimal strings.

Times are UTC Unix milliseconds represented as nonnegative JSON safe integers, within $0 \ldots 2^{53} - 1$. The zero-byte separator is an actual byte, not the two printed characters backslash and zero.

Parsing rejects duplicate object keys and invalid Unicode before JCS. This profile restricts JSON numeric values to safe integers; fractional and nonfinite values are rejected. The serializer preserves Unicode without normalization and orders properties by UTF-16 code units. Inputs also meet the action's typed constraints; decimal money should use explicitly typed integer minor units or canonical decimal strings, not ambiguous binary floating-point amounts. JCS is a byte-encoding rule, not schema validation, authorization, evidence verification, or a state transaction [48]. The complete schema, fixture keys, positive/negative cases, and cross-language signature vectors accompany the distribution.

MCP access authentication and KedgeFlow user delegation are separate bindings. Core protected MCP uses its specified resource access token; the KedgeFlow deployment authenticates the subject's DPoP-bound delegation on its admission/execution interface and independently authenticates the workload. It MUST document any negotiated token carriage used with MCP and MUST NOT treat an arbitrary identity header as authenticated context. The reference proxy includes a narrow pinned-Ed25519 access-token/DPoP verifier with verified SPIFFE mTLS context. Its test issuer and replay cache are bounded examples; enterprise issuer discovery, key rotation, distributed replay prevention and workload authorization require deployment integration. Its negotiated DPoP authentication is a KedgeFlow profile choice, not a claim that core MCP mandates that scheme.

7. Execution Authority and Resource Guarantees

The interfaces separate requesting, admitting, authorizing, and reporting an action. The execution contract now connects those exchanges to one bounded target attempt. It answers three distinct questions: is the proposal admitted, may this caller attempt it, and did the target effect occur? The following records make those distinctions precise; their safety properties depend on the declared resource and deployment contracts.

7.1. Proposal, Admission, and Execution Grant

The Agent Adapter freezes a proposal before requesting admission:

$$q = \langle id, p, i, a, r, \theta, d, x, h_{s_0} \rangle. \tag{2}$$

Here id identifies the proposal; $p = \langle u, w, j, d_{\text{del}} \rangle$ is the authenticated subject/workload/delegation context of Section 6.6; i is intent; a includes the canonical action and schema version; r identifies target resources and their incarnations; θ contains exact canonical parameters; d contains dependencies; and x contains declared context. The referenced observed state is s_0, identified by h_{s_0}. A digest binds the chosen representation. It does not establish that the observation is current or that the intent is correct.

The equations use $H(\cdot)$ for the digest required by the relevant record or value contract. Section 6.9 defines wire-profile digests; the review contract below identifies its raw-byte hashes. Sign_{sk} denotes a signature using private key sk; sk_C and sk_G are the Admission Controller and Grant Authority signing keys, respectively. These tuples summarize semantic bindings; the machine-readable schema specifies the actual fields and envelope bytes.

Admission Certificate (C_q). The Admission Controller evaluates the obligation set $\Omega(q, s_0)$ under policy Π_v at epoch v_Π. It verifies that accepted evidence discharges the required obligations and records its bindings in evidence manifest W; $H(W)$ binds that signed manifest:

$$C_q = \text{Sign}_{\text{sk}_C}\left(\langle H(q), v_\Pi, H(W), \Omega_{\text{discharged}}, H(\kappa), t_{\text{admit}}, t_{\text{expire}} \rangle\right). \tag{3}$$

$\Omega_{\text{discharged}}$ records the satisfied obligations; t_{admit} and t_{expire} record admission time and expiry. The evidence manifest records scope, provenance, freshness, and trust requirements. The signed execution constraints κ bind the guard and predicate scope, adapter descriptor and required level, allowed audience/completion mode, maximum lifetime, ownership epoch, and freshness rules at dispatch or commit.

Issuance may narrow these constraints but cannot broaden them. The certificate attests to the controller's evaluation under those rules; it does not prove semantic truth, reserve shared resources, or authorize a gateway. `Denied` and `NeedsEvidence` produce no certificate and no execution authority.

Execution Grant (G). The Grant Authority validates the certificate, applicable policy/evidence validity, requested scope, and any required reservation before issuing:

$$\begin{aligned} G = \text{Sign}_{\text{sk}_G}\big(\langle & gid, root_id, gen, H(q), H(C_q), \text{aud}, p, a, r, \\ & H(\theta), n, f, b, g_s, v_\Pi, e_{\text{scope}}, H(\kappa), t_{\text{issue}}, t_{\text{expire}}, \mu \rangle\big). \end{aligned} \tag{4}$$

gid identifies the grant; *root_id* identifies its execution root; and *gen* identifies the root generation. aud identifies the gateway audience; n is a single-use nonce; f is the root generation's fencing value; b references any authoritative reservation; g_s is the declared state predicate; e_{scope} is the authority-ownership epoch; and μ defines completion mode. The time fields specify issuance and expiry.

A grant binds the subject issuer and subject ID, workload SPIFFE ID, DPoP key thumbprint, delegation digest, exact proposal, resource incarnation, policy epoch, action, parameters, and approved constraints. The tuple p carries these four identity bindings in Equation 4; the machine schema expands them into individually validated fields. Changed parameters or a recreated target require a new proposal, admission, and authority decision.

7.2. One Authority Ledger for Issuance and Claim

Issuance and redemption must serialize against the *same* authoritative root-generation/nonce/fencing ledger. Separately ordered issuer and gateway stores still allow replacement to race with redemption; both operations must share one serialization boundary. The issuer assigns a stable issuance key to the admitted operation and budget; caller-chosen request IDs or new root IDs cannot multiply one admission. A gateway's atomic claim checks the current root generation and fence, active-grant pointer, ledger grant status, nonce, applicable revocation, and reservation in one transaction, then records the attempt and dispatch intent. All gateway replicas use this authority domain; loss of access prevents a new claim.

In the claim rule, R is the current root record addressed by $G.root_id$, and $L[gid]$ is the authoritative ledger record for a grant. Dotted names denote fields of the verified grant or root record. Successful claim records consumption and the attempt atomically with these checks:

$$\text{Claim}(G, attempt) = \begin{cases} \textsf{Claimed}, & \begin{array}{c} G.gen{=}R.gen \wedge G.f{=}R.f \\ R.active_gid{=}G.gid \wedge L[G.gid].status{=}\textsf{Unclaimed} \\ G.n \text{ unused} \wedge \text{authority and reservation valid} \end{array} \\ \textsf{Refused}, & \text{otherwise.} \end{cases} \tag{5}$$

An executable leaf root bounds one target action. Replacement or *transfer delegation* may atomically retire its unclaimed generation and create one narrower successor. *Partition delegation* instead allocates separate child roots under a non-executable composite saga root (Section 7.7). The parent cannot execute or spend an allowance already allocated to a child. The successor's principal, audience, parameters, and obligations are separately validated; delegation cannot multiply the allowance.

If a claim wins first, retirement is not evidence that the target did nothing. Claimed, dispatched, pending, and unknown roots cannot receive replacement authority merely because their grant expired or was marked retired. A confirmed effect spends the root's budget. A claimed attempt permits a successor only after authoritative proof of no effect, a barrier excluding late effects, fresh admission, and validation of the remaining budget. A resource-enforced transfer must prove equivalent no-effect and fencing conditions. The default is to retain the consumed allowance and reconcile.

A durable claim prevents a second redemption; it does not alone create exactly-once physical execution. A process can fail between logging dispatch intent and receiving the target response. The Resource Adapter must declare target idempotency/correlation support and recovery behavior. Without proof that redispatch is safe, a recovered gateway queries status and leaves the attempt unknown. It does not release the nonce or create a new grant to conceal uncertainty.

7.3. Shared Invariants Require Reservations and Writer Coverage

The Resource Registry reports observations and dependency descriptions; it does not arbitrate concurrent capacity. Where policy depends on a shared budget or conflict set, the Grant Authority must acquire authoritative reservations atomically with issuance and retain them through unresolved attempts. Two agents cannot each spend the same unreserved allowance based on a stale registry view.

A reservation in the control ledger protects only the actors that honor it. To claim a resource-state invariant, enforcement must cover the *complete predicate* and every writer capable of invalidating it, including human operations, scheduled jobs, and other controllers. This can require a resource-side transaction, conditional mutation, or fencing protocol spanning the relevant state. A conditional write to one record is insufficient when the policy also depends on another unprotected record. When this coverage is absent, the implementation must report the narrower guarantee rather than claim cross-agent or global state safety.

7.4. Adapter Guarantee Levels (A / B / C)

The A/B/C overview in Section 3.1 distinguishes atomic enforcement, pre-dispatch revalidation, and deferred-effect observation. The contracts below state their precise limits. Each level applies to an *action path and declared predicate*, not to a vendor product as a whole. The Resource Adapter descriptor identifies the checked fields, writer coverage, target primitive, and failure/recovery semantics. Write $s_{\text{precommit}}$ for the target state at the atomic guard check.

Level	Mechanism	Supported Claim and Limit
A: Atomic predicate enforcement	The resource checks the entire declared guard and commits the authorized mutation indivisibly, using a suitable conditional write, transaction, or resource-enforced fence.	Successful commit implies $g_s(s_{\text{precommit}}) = \textsc{True}$. A false guard prevents that mutation. Other authorization, availability, or target errors can still prevent commit; equality to the whole observed state is not required unless included in the guard.
B: Pre-dispatch revalidation	The gateway queries the target, checks the guard, then dispatches through an API lacking an atomic guard-and-mutation primitive.	The guard was true at the observed check. A later concurrent write can invalidate it before effect. The measured check-to-dispatch interval is exposure information; it is not an unconditional bound on target commit or a state-safety guarantee.
C: Deferred-effect observation	The target accepts work whose effect occurs later; the adapter correlates jobs/events and queries authoritative completion state.	Dispatch authorization and observed outcome are distinguishable. Reconciliation can detect an unwanted effect; it does not retroactively make it safe. Compensation is a separate authorized action and may be impossible or fail.

Table 9: Resource Adapter guarantees. Asynchronous paths need an additional resource-side contract to support a commit-time guard.

The Level A Invariant

For a covered Level A mutation, successful commit implies that the declared guard was true at its atomic check-and-commit point. This is a one-way safety condition, not an "if and only if" promise of execution and not a promise that all resource state equals a prior snapshot. Levels B and C do not inherit this invariant.

7.5. Completion Modes and Outcome Lifecycle

DispatchBound authorizes initiation of the exact request by the gateway within the signed validity window, under the adapter's declared target-acceptance semantics. The gateway rechecks applicable grant, policy, and evidence validity immediately before dispatch; a prior claim is not permission for an indefinitely delayed call. A pre-send check and request deadline cannot prove that a remote target accepted the request before expiry. If policy requires that deadline at target acceptance, the adapter must supply a target-enforced expiry check. A target acknowledgment is a dispatch result, not proof that provisioning, payment settlement, or another physical effect completed.

CommitBound requires an authoritative terminal observation and, when a state guard must hold at a later effect, resource-side revalidation and fencing through that effect. An asynchronous API lacking that contract cannot offer a guarded CommitBound profile merely by polling afterward.

Evidence expiry, policy revocation, cancellation, and claim/reservation retention must have explicit meanings for each mode. Revocation prevents new claims once recorded in the authority domain; cancellation of already claimed or dispatched work requires the target's declared barrier and cannot be inferred from a control-plane update.

The lifecycle is `Proposed` → `Evaluating` → `Admitted` → `Granted` → `Claimed`, followed by a resource result. Evaluation can instead return `Denied` or `NeedsEvidence`. A verified pre-dispatch refusal records `NotDispatched`; accepted work records `Dispatched` or `Pending`; an authoritative terminal effect records `Committed` or `Failed`. Loss of a decisive response records `Unknown`.

The Outcome Service queries the existing attempt through the gateway/adapter and preserves supporting observations. It does not infer commit from agent text, interpret a missing receipt as no effect, or authorize blind retries.

The wire profile uses lower-case receipt states. `not_dispatched` records a verified dispatch refusal; `failed` and `compensation_failed` record failures with error and effect evidence, without implying zero effect. Only `fenced_no_effect`, backed by resolution evidence and the required barrier, can support restoring an allowance. Refusals before a grant/attempt exists are interface diagnostics rather than fabricated attempt receipts.

7.6. Hardware-Bound Review Witness

Let B be the immutable review record, with RP denoting the relying party and EP denoting the Evidence Provider:

$$B = \langle H(q), v_{\Pi}, u_{\text{reviewer}}, H(T), H(\text{Render}_T(\theta)), H(\theta), t_{\text{review}}, \text{expiry}, \text{verdict}, n_{\text{review}}, \text{RP} \rangle. \tag{6}$$

T is the registered template, u_{reviewer} is the authenticated reviewer (who may differ from the initiating subject when policy permits), and time and nonce are issued or checked by the trusted review service. The authoritative panel renders the exact canonical parameters through T, together with the bound

resource/action identity and policy epoch. Agent prose cannot replace these fields. The renderer must reject omitted, hidden, substituted, or ambiguously formatted execution fields.

Let A_{user} denote the received WebAuthn assertion. In this review contract, $H(T)$, $H(\text{Render}_T(\theta))$, and $H(A_{\text{user}})$ are ordinary SHA-256 hashes of the exact declared template, rendered-action, and assertion-record bytes, without KedgeFlow domain prefixes. The proposal and parameter digests retain their wire-profile definitions.

WebAuthn does not directly sign arbitrary JSON using a user key. The review service sets the SHA-256 digest of the canonical `ApprovalPayload` B, encoded as unpadded base64url, as server challenge c_{review}. The bound RP ID, reviewer, nonce and transaction fields supply the profile context. The authenticator signs its authenticator data and the hash of the client data, which contains that challenge [8]. Write W_{hitl} for the provider-signed review witness and $\|$ for byte concatenation:

$$\begin{aligned} c_{\text{review}} &= \text{base64url}(\text{SHA256}(\text{JCS}(B))), \\ A_{\text{user}} &= \text{WebAuthnSign}(\text{authenticatorData} \,\|\, \text{SHA256}(\text{clientDataJSON})), \\ W_{\text{hitl}} &= \text{Sign}_{\text{sk}_{EP}}\langle B, H(A_{\text{user}}), \text{verifiedClaims}\rangle. \end{aligned} \tag{7}$$

The WebAuthn client-data hash uses the original received bytes, without JCS encoding or a KedgeFlow prefix. The Evidence Provider (EP), whose signing key is sk_{EP}, verifies the expected challenge, credential signature, RP ID, origin, user verification, credential enrollment, expiry, replay prevention, and the configured authenticator assurance policy. The Admission Controller checks the provider's trust and the exact proposal/policy/template/rendering bindings. A click record, natural-language approval, assertion for another challenge, or unbound signature is `Implicit/Unverified`, not a discharge of a hardware-bound review obligation.

User verification alone does not prove a nonexportable hardware key: synchronized passkeys can satisfy WebAuthn user verification. A hardware-bound profile therefore requires enrollment and attestation evidence meeting the deployment's device policy. The signature also does not prove that a compromised browser displayed the correct transaction. The trusted renderer, review origin, credential enrollment, and client integrity are explicit trust assumptions. The witness establishes verified approval of a bound record under these assumptions; it does not establish the action's business correctness.

7.7. Hierarchical Saga Execution Roots

A workflow DAG $D = (V, E)$ contains action steps V and dependency edges E. Its composite root is

$$R_{\text{saga}} = \langle rid, gen, H(D), p, \mathbf{B}, \mathbf{B}_{\text{free}}, \text{children}, \text{status}, f\rangle. \tag{8}$$

Here *rid* identifies the composite root; *gen* and f identify its current authority generation and fence; and children and status record its allocations and lifecycle. $\mathbf{B}$ is a nonnegative vector of typed allowances, such as action count, capacity units, monetary units, and compensating-action count. Write $\mathbf{B}_{\text{initial}}$ for the initial allowance and $\mathbf{B}_{\text{free}}$ for its unallocated balance. Units and per-action costs are policy-defined; incomparable units cannot be added.

The concrete KF-JSON-0.2 root encodes one declared unit with forward and compensation allowances, a restricted instance of this vector model. The root is a workflow allocation record, not an executable wildcard grant. Each child still needs an exact proposal, certificate, grant, caller binding, and target guard. DAG edges express prerequisites, not an implied permission to run all descendants.

The Grant Authority implements `delegate(gid, childCertificate)` against the current parent generation. In one authority transaction it verifies the parent's remaining delegation rights, the child's

narrower identity/audience/resource/action scope, admitted constraints, predecessor status, and cost; creates a unique child allocation; and subtracts that allocation from the parent's free budget. Parent and child cannot concurrently spend the same allowance. Recursive delegation applies the same transaction, including a persisted deduplication key; an A2A task or a new request ID cannot create a second allocation.

For a root with immediate child allocations A_i, conservation is componentwise. Each child term recursively aggregates its entire subtree, including that child's exclusive local balances and descendant allocations; subtrees are disjoint. Descendants are not added again to this sum. Compensation allocation is owned by exactly one such subtree or the root's free balance:

$$\mathbf{B}_{\text{free}} + \sum_i \left(\mathbf{B}_{i,\text{available}} + \mathbf{B}_{i,\text{held}} + \mathbf{B}_{i,\text{spent}}\right) = \mathbf{B}_{\text{initial}}. \tag{9}$$

Available means an allocated but unconsumed allowance, held means an allowance reserved by an unresolved attempt, and spent means an allowance definitively consumed under the target cost contract. A held allowance includes claimed, dispatched, pending, and unknown attempts. A confirmed effect moves its allocation to spent. Unclaimed allocations may be reclaimed only by an atomic retirement; claimed allocations require authoritative no-effect evidence and a barrier against late effects before any refund or successor. Generation replacement never resets the initial budget. Fees or other charged failed attempts are accounted for according to the registered target contract rather than silently refunded.

Compensating Action Descriptors. A CTD registers an inverse-action reference, applicability predicate, parameter derivation, dependency order, and execution deadline, or explicitly declares `none`. The inverse is a semantic compensation, not necessarily a mathematical inverse: refunds may incur fees, re-created infrastructure may have a new identity, and some operations cannot be reversed [47]. An irreversible action cannot promise rollback by inventing an inverse descriptor.

Workflow admission allocates forward and compensation allowances separately within $\mathbf{B}_{\text{initial}}$. A compensating descriptor $\bar{q}$ is pre-authorized only as a bounded action template and budget allocation. Before compensation, the Outcome Service obtains a fresh exact proposal/certificate/grant under current identity, policy, evidence, and target state. It schedules inverses in the registered reverse dependency order and records each result through the same gateway. It cannot mint grants, bypass guards, or reuse the failed child's nonce. Insufficient compensation allowance or an expired applicability deadline yields an explicit unresolved workflow, not an unrestricted undo operation.

Failure and uncertainty. If a downstream child is authoritatively known to have produced no effect, the Outcome Service may coordinate compensation of confirmed predecessor effects. A timeout or `OUTCOME_UNKNOWN` alone is not sufficient: a late forward effect could race its inverse. Unknown children retain allocations and require authoritative effect resolution and any necessary resource-enforced late-effect barrier before an inverse whose correctness depends on that outcome. A barrier excluding future effects does not establish whether an earlier effect occurred. A failed compensation remains visible, with its own consumed allowance and outcome. Saga completion therefore distinguishes `Completed`, `Compensated`, `CompensationFailed`, and `Unresolved`; it does not promise atomic rollback across independent resources.

Bound automated recovery for legacy targets. A Level C webhook or legacy API may have neither authoritative operation lookup nor enforceable cancellation/fencing. Its operating record **MUST** name retained correlation identifiers, available status evidence, polling/backoff limits, an escalation deadline

and the accountable resource owner/on-call team. At that deadline, open a human incident and apply its declared limits on automatic recovery. Specify which effectful actions cease and whether bounded query-only observation continues; unknown effects never permit blind redispatch or an unsafe inverse. Preserve `Unresolved`, consumed allowances and reservations requiring resolution. Elapsed time, a site reliability engineer (SRE) acknowledgment or administrative incident closure cannot establish no effect, refund authority or make an inverse legal. Compensation still needs fresh admission, accepted business constraints, registered applicability/dependency/deadline checks and the required outcome/barrier evidence. A separately admitted mitigation safe under uncertainty may proceed within its own budget and guard; it cannot erase the original unknown. If the action's required recovery cannot be supplied, do not enable that protected action.

7.8. Pre-Flight Admission Leases and Just-in-Time (JIT) Evidence

Admission can pipeline slow evidence acquisition without weakening final checks. A signed provisional lease binds the user/workload/delegation context, exact frozen-proposal scope, policy and descriptor digests, discharged static obligations, evidence expiry, and lease expiry. KF-JSON-0.2 leases require a non-null exact proposal digest; template-only leases are outside this profile. It carries no execution nonce, target credential, reservation, or dispatch authority. Identity membership, licensing, and role-based access control (RBAC) are only *relatively* static: revocation, policy activation, descriptor replacement, or delegation expiry invalidates the corresponding lease before use.

Immediately before final admission and grant minting, the Admission Controller ingests the fast-decaying witnesses required by the CTD and policy: current cluster capacity, lock availability, or other live state. It revalidates the lease's assumptions and signs an exact C_q only after all obligations are discharged. The Grant Authority acquires the required authoritative reservations. The gateway rechecks dispatch-time validity, and Level A targets still check their declared predicate at commit. A JIT observation reduces staleness exposure; it does not close the observation-to-mutation race by itself.

Independent evidence providers and disjoint conflict domains may be evaluated concurrently. The implementation records acquisition time, observed resource version, freshness bound, and the phase at which validity must hold. It does not infer freshness from a recent signature or use lease renewal to extend an expired underlying witness.

7.9. Conflict-Domain Layout in an etcd Authority Ledger

An etcd v3.6 deployment groups authority records by conflict domain, for example `/kf/domains/orders-db/roots/r1`, with child keys for grants, nonces, reservations, attempts, and audit outboxes. A *durable outbox* holds persisted events awaiting publication, so a failed or delayed report does not erase the underlying transition.

Claim uses one transaction comparing the current root generation/fence/active grant, unclaimed grant record, unused nonce, current policy/ownership epoch, and required allocation records. Its success branch records the consumed nonce, claim, dispatch intent, and publication record together. Issuance and transfer use the same authority records and compare their revisions; gateway-local caches cannot substitute for this transaction [30, 31].

Domain-local keys reduce unnecessary application conflicts and permit independent requests to pipeline. Prefixes do *not* shard etcd's Raft log: a single cluster still has one replicated ordering and finite leader, disk, and network throughput [55]. A shared parent saga budget or cross-domain reservation can remain a serialization point. Genuine partitioning requires separate authority clusters, an explicit

domain-to-owner mapping with ownership epochs, and a transfer barrier that excludes both old and new owners from concurrent issuance. A workflow spanning clusters needs a declared coordination protocol; separate successful transactions do not constitute one atomic global reservation. The bounded prototype uses one real three-member cluster and does not claim cross-cluster atomicity.

8. Implementation Blueprint and Worked Trace

A KedgeFlow deployment connects existing agent workflows to an explicit authority path. Models, planners, memory, and orchestration stay in their existing harness. The Agent Adapter supplies exact-action admission, while the Governed Reverse MCP Execution Gateway replaces the protected tool endpoint. The gateway owns claim and credential exchange; a Resource Adapter implements the target-specific guard and outcome contract. The worked trace below is an implementation blueprint, distinct from the bounded executable verification reported in Section 8.4.

8.1. Integrate the Path Before Moving Privilege

1. **Define the surface.** Inventory actions, target incarnations, alternate writers, credential routes, consequence tiers, conflict domains, and completion semantics. Register CTDs and compensation descriptors; define policy obligations, trust roots, and review templates. Choose a narrow action path with verifiable writer coverage.
2. **Assemble the nine logical roles.** Attach an Agent Adapter to the existing tool boundary. Assign Policy Authority, Resource Registry, Admission Controller, Grant Authority, and Outcome Service responsibilities explicitly. Connect Evidence Providers. Deploy the gateway and Resource Adapter with one shared issuance/claim authority domain. Roles may share a service, while their authority and credentials remain separated.
3. **Exercise isolated enforcement.** Test missing grants, exact binding, hardware review, credential downscoping, stale witnesses, concurrent claims, false guards, and unknown outcomes. Shadow decisions identify integration gaps but do not establish mediation.
4. **Cut over protected privilege.** Remove equivalent target mutation credentials from agent runtimes; make target authorization and available network restrictions enforce the declared gateway path. Account for administrators, background controllers, alternate SDK routes, and delegated agents. Expand only after bypass, recovery, and target-state tests pass.

8.2. Local Developer Deployment and the Confused Deputy

A *Governed Loopback Gateway* is a local deployment of the Execution Gateway role. It lets a developer retain an existing CLI, permitted workspace operations, and local tool implementations while routing registered protected effects through independent admission and enforcement. The Control Plane may run on the workstation or as a shared service; it must be outside the agent's authority to alter or impersonate. Locality determines placement, not the trust boundary. Figure 4 shows the required separation.

The authority-confusion path. An agent becomes a confused deputy when untrusted context induces it to exercise authority that the caller did not intend to delegate to that action. The authority can be an inherited cloud token, an SSH agent, a writable deployment file, a privileged socket, or a remote tool service with excessive permissions. A remote MCP server can enforce substantial authorization; merely relocating a privileged executor neither establishes nor defeats the boundary. Existing agent permissions, sandboxes, and trusted credential proxies already address parts of this risk [56, 57, 58]. KedgeFlow specifies how such containment connects to exact proposals, independently verified evidence, conserved grants, target guards, and outcome recovery.

The Agent Adapter freezes q from Equation 2. Authentication identifies the subject/delegation and provisioned workload; it does not establish that an uncompromised operator intended each action or that the model is honest. Independent admission decides whether the exact proposal satisfies the declared policy. The grant binds $H(q)$, $H(\theta)$, identity, policy, audience, resource incarnation, and one allowance. Substitution and replay are refused under the trusted enforcement contract. An injected proposal can still be admitted by an incomplete policy or deceptive evidence; a valid signature is not a proof of correct intent.

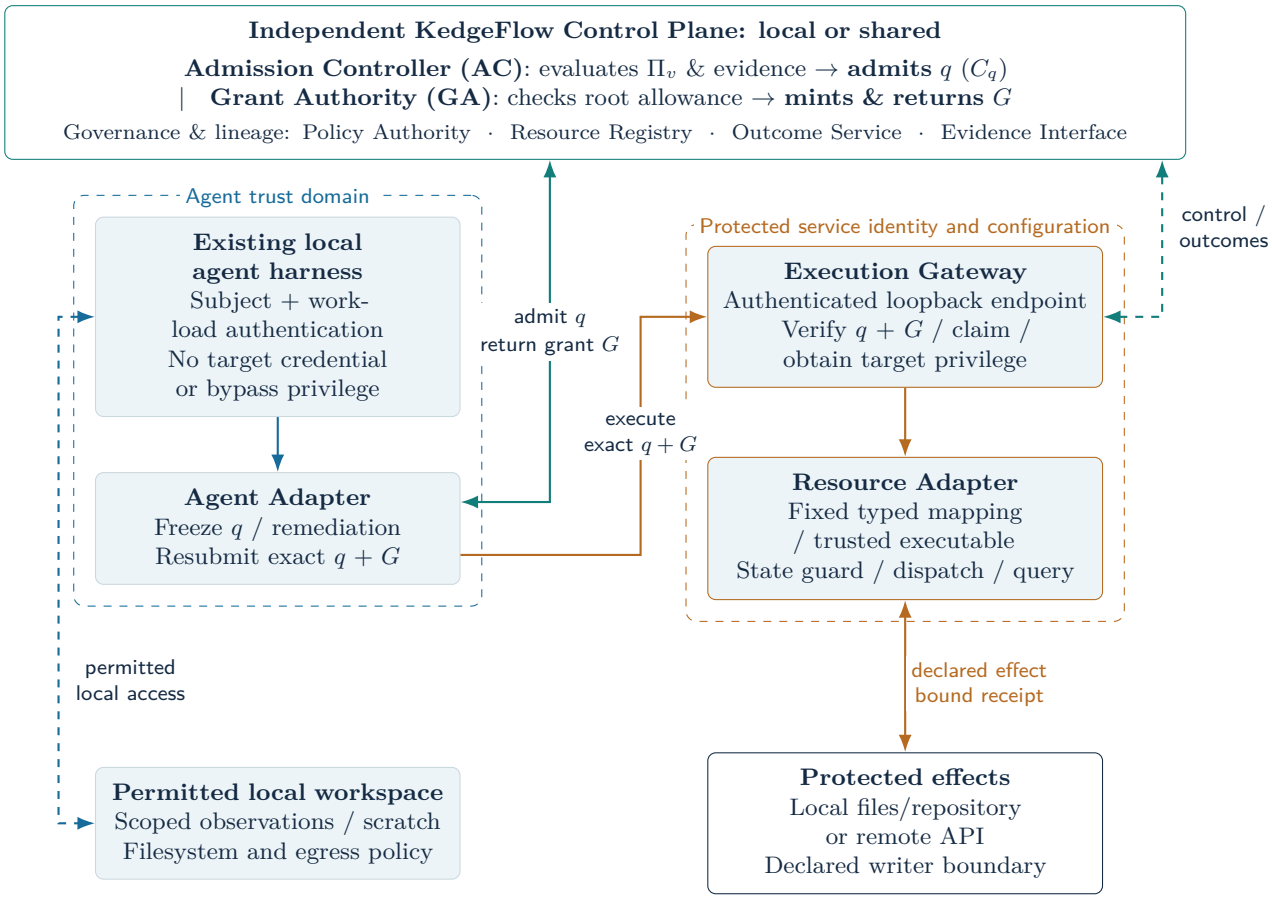


Figure 4: A local placement of existing KedgeFlow roles. The agent's permitted workspace path cannot reach protected secrets, gateway configuration, or equivalent mutation authority. Loopback access is authenticated; direct shell, file, socket, SDK, and delegated paths to protected effects must be denied or separately mediated. The independent control plane need not be hosted in a cloud.

Establish the local boundary before routing tools. The agent **MUST NOT** read protected-target credentials, modify gateway code/configuration or trust stores, attach to its privileged process, or invoke an equivalent privileged helper. A distinct protected service identity, or an independently enforced sandbox with equivalent separation, supplies this property. Moving a token into a keychain alone does not: custody depends on the item's actual access policy and the identities permitted to use it [59]. Inventory environment/files, SSH and credential-agent sockets, cloud metadata, browser sessions, database connections, sudo rights, container-management sockets, and automation triggered by repository edits. Access to a Docker daemon can confer host-level privilege [60].

MCP configuration routes configured MCP tools; it does not universally intercept a harness's built-in shell, file editor, SDK client, subagent, or extension. Claude Code and Codex document separate permissions for those surfaces [56, 57]. Each protected path needs a native hook or wrapper plus enforced denial of bypass, or must be disabled for that scope. The agent can retain writable scratch space without obtaining authority over protected deployment files. A human using independent administrative authority remains a separately declared writer; the paper does not imply that ordinary interactive workstation use is entirely governed.

Contain shell authority at the OS boundary. Routing MCP calls through loopback leaves a bypass if a built-in terminal can read `~/.aws/credentials`, inherit an equivalent token, or connect to a Docker daemon socket such as `/var/run/docker.sock`. The OS/service boundary must restrict credential files, host mounts, process attachment, privileged sockets and unapproved target connections, and remain effective in child processes. Permitted shells and scratch operations can run inside that boundary.

A configured macOS App Sandbox or Linux bubblewrap sandbox is a possible mechanism; its actual permissions and launch configuration determine coverage [61, 62]. Linux cgroups control resource use and do not by themselves isolate ordinary credential files [63]. Rootless Docker reduces host-root daemon privilege, but does not establish gateway-exclusive secret custody or safe bind mounts [64, 65].

Test credential reads, socket connections and equivalent target calls from the harness's built-in terminal and its descendants before enabling protected dispatch. This workstation boundary is required deployment work, not a capability supplied by the reference loopback proxy.

Authenticate the local endpoint. A local HTTP binding **MUST** authenticate the caller and pin the gateway audience, authority and path. It **SHOULD** bind only to the selected loopback address; a port on localhost is reachable by other local processes. Validate Host and Origin, deny unconfigured browser origins, protect service discovery/configuration from substitution, and retain proof-of-possession and workload authentication. MCP explicitly requires invalid supplied Origins to be rejected [66].

The reference CLI uses HTTPS with verified workload client certificates. An illustrative endpoint is `https://127.0.0.1:8989/mcp`, with a certificate and authenticated URL configured for that address. Plaintext test fixtures are not its deployment profile. Browser-based human review uses the Evidence Provider's separately configured origin/RP binding.

Preserve useful local operations without inventing a read exemption. An authorized bounded observation may avoid mutation admission while still obeying filesystem, confidentiality, egress, and output-handling policy. Returned content remains untrusted input to the model. Arbitrary shell commands are not a dependable read-only category: even `git status` can refresh the index, and configured helpers can execute programs [67, 68]. Tier 0 is assigned to the actual registered behavior,

not to a command's familiar name. Admission latency and integration cost must be measured; loopback placement does not establish wire-speed operation or zero workflow friction.

For protected local execution, a Resource Adapter **MUST** constrain the complete effect inputs: fixed action mapping, executable provenance, arguments, environment, working directory, scripts/hooks, target identity, and relevant base state. A privileged shell that executes an agent-controlled string is not a complete adapter. Mutable scripts, configuration, symlink resolution, or hooks must not substitute a different action after review. An illustrative Git-publish contract binds repository and remote identity, exact new commit/object IDs, destination refs and expected old IDs; the adapter declares which compare/commit and recovery conditions the actual server enforces. This is a descriptor-design example, not an implemented Git adapter or an unqualified Level A claim.

Use the existing admission and execution interfaces. A missing grant returns the negotiated `NEEDS_EVIDENCE` RemediationContract from Section 5.3; the local pattern introduces no second error shape. The Agent Adapter validates the trusted admission endpoint and freezes the exact proposal. Policy determines whether human review is required. A terminal `yes` or an agent-generated confirmation cannot discharge a hardware-bound review obligation; that obligation uses the verified canonical WebAuthn flow. A separately defined terminal-review profile would need protected rendering, authenticated reviewer identity, signed action binding, and explicit assurance limits.

After C_q and G are issued, the gateway verifies and atomically claims the grant, then obtains a narrowly scoped target credential through the configured broker or uses a protected local capability declared by the adapter. It does not return a raw key or general shell privilege to the agent. Where a target cannot enforce exact credential scope, the trusted gateway/adapter remains the action enforcement point and that limit is explicit. Unknown outcomes retain consumed authority and require reconciliation. Failure to obtain or validate required admission/claim authority blocks new protected effects; it does not authorize fallback to the developer's ambient credential. An Outcome Service outage can instead retain pending records in durable outboxes under the declared availability contract.

Dimension	Assumed Ambient Local Path	Remote MCP Service	KedgeFlow Local or Remote Profile
Target authority	Agent can use inherited target privilege.	Custody and scope depend on server/issuer policy.	Protected enforcement identity; claim before target privilege is exercised.
Exact-action admission	Inherited privilege alone does not bind reviewed action.	Can enforce exact-action policy; MCP access authorization alone does not supply this contract.	Verified C_q, bound G, and coordinated allowance claim.
Native tool coverage	Depends on OS, runtime and target controls.	MCP routing does not mediate unrelated native paths.	Every protected path denied or mediated, including local shell/files/sockets.
State and outcomes	Depend on actual resource mechanisms.	Depend on server and target mechanisms.	Declared guard/completion level, consumed unknown attempts and reconciliation.

Table 10: Location does not determine security strength. The ambient column is an explicitly assumed authority path, not a characterization of all local agents. A remote service can implement the same KedgeFlow contracts. No latency, cost, or efficacy ranking is inferred.

Deployment acceptance. An implementation **MUST** demonstrate that alternate native tools, secret/socket access, gateway tampering, forged identities, browser/Host abuse, mutable executable substitution, and read disclosure cannot escape the declared profile. The supplied reference proxy tests grant/claim behavior and named HTTP endpoint checks; it does not provision a workstation sandbox, OS-keychain broker, Claude Code/Codex interception layer, or native Git/filesystem adapter. Those remain deployment work. No change to KF-JSON-0.2 is required by placing the existing roles locally.

8.3. Worked Trace: A Governed Kubernetes Node Drain

An engineer asks an operations agent to drain `node-1` serving `orders-db`. Another agent is considering `node-2`. The declared maintenance policy requires enough capacity for two nodes to remain available, a recent capacity witness, and human approval of the exact node and options. With three available nodes, at most one maintenance removal is permitted. The SQL record's `available_slots` below represents that coordinated maintenance allowance, not a live measurement of Kubernetes availability. These are example policy requirements, not universal KedgeFlow defaults.

A platform maintenance controller owns a SQL reservation record and coordinates every maintenance writer in the declared scope. It exposes a governed MCP tool `maintenance.drain`. Its CTD declares a Level A *reservation step* and a separate Level C *physical drain step*. Kubernetes node cordoning and pod evictions are multiple API operations; they are not one SQL transaction. The Eviction API respects configured PodDisruptionBudgets, while acceptance of an eviction is not completion of an entire drain [69, 70]. This distinction is essential to the trace.

1. Negotiate the governed MCP profile. The agent's existing MCP client addresses the gateway and sends the protocol version and client capabilities on the request. This complete illustrative request contains no execution grant:

```
{
  "jsonrpc": "2.0",
  "id": 41,
  "method": "tools/call",
  "params": {
    "name": "maintenance.drain",
    "arguments": {"node": "node-1", "expected_version": 14},
    "_meta": {
      "io.modelcontextprotocol/protocolVersion": "2026-07-28",
      "io.modelcontextprotocol/clientCapabilities": {
        "extensions": {
          "io.kedgeflow/governed-mcp": {"profile": "KF-MCP-0.1"}
        }
      }
    }
  }
}
```

The profile is a proposed extension. The gateway authenticates MCP access, the human delegation, and the workload through its configured verifier; it does not accept an agent-supplied subject header. It classifies the registered descriptor rather than trusting a server's tool annotation to downgrade consequences. Authorized read tools pass through with redacted telemetry; protected writes require the authority path.

2. Return non-authorizing remediation. The missing grant produces JSON-RPC error code `-32001`, message `NEEDS_EVIDENCE`, and a structured `RemediationContract` as `error.data`. The response binds the tool/action and canonical parameter digest, names the unmet policy and JIT state obligations, and provides a configured admission endpoint, expiry, and grant carriage.

Before the full proposal is frozen, `proposal_digest` is explicitly null: the response asks the adapter to freeze and admit a proposal. Once q exists, the contract binds $H(q)$ and policy/descriptor context through the registered admission flow. Appendix D supplies a complete signed offline remediation fixture and schema. The lightweight proxy's initial unsigned response is authenticated by its transport and documented as a narrower implementation surface.

The Agent Adapter freezes q with the tool arguments, exact target incarnation, subject issuer/ID, workload SPIFFE ID, DPoP key thumbprint, delegation digest, and registered descriptor. A new request ID on resubmission is transport correlation; the authority ledger deduplicates issuance using the proposal's admitted operation identity.

3. Obtain exact human review and live evidence. A provisional lease can discharge the still-valid static policy obligations. The Resource Registry selects the immutable drain template; the authoritative review panel renders the canonical node and version, operation identity, and policy epoch. The reviewer approves a WebAuthn challenge bound to $H(q)$, template/rendering digests, verdict, nonce, and expiry. The trusted Evidence Provider verifies the assertion and hardware enrollment requirements and emits W_{hitl}. An approval of prose alone is rejected.

Immediately before final admission, a provider supplies a capacity witness scoped to `orders-db` and observation version 14. The Admission Controller validates freshness, trust, identity, policy, and the review binding, then signs C_q with the declared constraints. Neither the lease nor the live observation reserves spare capacity.

4. Allocate saga authority and reserve capacity. The workflow uses separate grants for reserving maintenance capacity and performing the physical drain. Its composite root contains separate allowances for those steps and any supported compensation. The Admission Controller admits each exact child; the Grant Authority atomically delegates its allowance from the current root generation.

Competing maintenance requests share the `orders-db` conflict domain. Issuance reserves the last permissible maintenance slot in the authority ledger against all participating writers; Step 6 records the reservation's target-side effect. The reservation-step grant binds the fence $G.f$, exact parameters, active generation, nonce, adapter descriptor, and SQL predicate. The physical drain child is issued only when its predecessor outcome and fresh obligations permit it.

5. Resubmit, claim, and exchange credentials. The adapter resends the same `tools/call`, carrying unpadded base64url of the complete signed grant envelope in `Mcp-KedgeFlow-Grant`. The gateway validates exact identity/action/parameter/audience/time bindings and atomically claims the active root generation, nonce, fence, and allocation. Only then does the credential broker process an RFC 8693 token exchange with the human's delegation as subject and the authorized workload as actor. Target audience, scopes, and lifetime are restricted by broker policy to the claimed action. AWS, database, and Kubernetes credentials require their declared provider-specific bindings; a token-exchange response is not automatically an AWS STS session. The gateway strips the inbound authorization, DPoP, identity, and grant headers before upstream dispatch.

6. Commit the Level A SQL reservation. The target transaction locks the shared maintenance budget and node intent, validates their incarnations/versions, active reservation and fence, and the entire registered predicate, and then updates both records. An implementation uses serializable isolation or equivalent explicit locking with all competing writers following the same contract. The transaction is logically:

```
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
SELECT available_slots, version, fence FROM maintenance_budget
  WHERE domain = 'orders-db' FOR UPDATE;
SELECT version, incarnation, state FROM maintenance_intent
  WHERE node = 'node-1' FOR UPDATE;
-- Validate complete guard, reservation ownership and signed grant fence.
-- If any check fails, ROLLBACK and report STATE_GUARD_VIOLATION.
UPDATE maintenance_budget
  SET available_slots = available_slots - 1, version = version + 1
  WHERE domain = 'orders-db' AND version = :budget_version
    AND available_slots >= 1 AND fence = :grant_fence;
UPDATE maintenance_intent SET state = 'reserved', version = version + 1,
  attempt_id = :attempt_id WHERE node = 'node-1'
    AND incarnation = :incarnation AND version = :node_version;
-- Require the expected affected-row count for both updates; otherwise abort.
COMMIT;
```

This is transaction pseudocode, with application validation and aborts explicitly shown. It demonstrates a resource-side atomic boundary; it is not a claim that ordinary SQL comments enforce a guard. The signed receipt confirms only reservation metadata. Neither the historical capacity witness nor the physical Kubernetes state participates in this SQL commit.

7. Dispatch and reconcile the physical drain. The drain controller consumes the separately admitted child grant and obtains a narrowly scoped Kubernetes credential. It persists an operation identity, validates the reservation/fence, cordons the node, and requests evictions through the Eviction API. It rechecks the required target conditions before each consequential step according to its registered contract. Other schedulers, autoscalers, failures, and privileged writers must be accounted for if a broader availability invariant is claimed. A Level C adapter correlates accepted requests and observes actual pod termination and workload availability. Polling afterward provides outcome evidence; it does not manufacture an atomic commit-time cluster guard.

8. Handle failure and compensation without inventing rollback. If the SQL guard is false, no reservation effect occurs and the result is `STATE_GUARD_VIOLATION`. If a later drain child is proven not to have dispatched, the Outcome Service may coordinate release of the confirmed reservation using its registered inverse, fresh admission, and compensation allocation. If eviction or drain status is unknown, the root remains held; release or uncordon requires authoritative operation status and a barrier excluding late drain effects. Uncordoning does not restore evicted workloads by itself. The final report distinguishes reservation commit, accepted eviction, physical drain completion, compensation failure, and unresolved state.

8.4. Executable Reference Surface

The primary reference runtime is the Rust `kedgeflow` crate, version `0.9.0-alpha.1`, with executable `kf-proxy`, in `openkedge/code/kedgeflow`. The frozen paper pins its implementation baseline to commit `0a06d00`. The Rust gateway ports the earlier Go execution prototype; the language-independent `KF-JSON-0.2` schema and all twenty-two fixture files are byte-identical across the two repositories.

The Rust gateway supports authenticated discovery, a negotiated `tools/call` subset, authorized registered reads, missing-grant remediation, exact grant verification, atomic etcd claims, postclaim RFC 8693 credential exchange, optional grant-reference retrieval, and query-only outcome reconciliation. Its executable profile accepts one nondelegated resource, one `version_eq` predicate, a Level A fencing contract, and dispatch-bound completion. Unsupported methods, batches, notifications, delegated grants, multiple resources/predicates, saga execution and commit-bound validity are refused. This is a bounded execution slice, not a complete implementation of every architectural role or MCP transport profile.

The target test adapter executes an atomic typed version predicate against a protected mock record. It does not run SQL, a Kubernetes cluster, an enterprise identity provider, a hardware authenticator, or a complete saga engine. A narrow pinned-Ed25519 OAuth/DPoP verifier and verified SPIFFE mTLS context exercise caller binding; trust provisioning and distributed proof replay protection remain deployment obligations. Trusted library ports support authority publication and issuance, but the executable exposes no public admission or grant-issuance service. Production registry, policy, admission, issuance, provider, outcome-worker and target integrations remain implementation work. Section 10 separates the executed artifact checks from these deployment gates.

The Go artifacts remain supplementary experiments in `acp/prototype` and `acp/services`, with their own retained verification records. The Go Admission Controller and HITL Evidence Provider exercise a separate capacity-reservation profile; they have not been ported to Rust or demonstrated as an end-to-end deployment with the Rust gateway. Their existence does not fill those missing Rust services.

The supplementary Go Admission Controller uses a pinned Cedar engine and authenticates signed proposals, descriptor scope, parameters, provider identity, witness freshness and canonical review. Its obligations return `Satisfied`, `Violated` or `Unknown`; only all-satisfied evidence plus a diagnostic-free policy permit yields signed C_q. The Go HITL Evidence Provider exposes challenge, deterministic review and assertion-verification endpoints, supports Ed25519 and the Elliptic Curve Digital Signature Algorithm (ECDSA) with the P-256 curve and SHA-256 (ES256), and produces the schema's provider-signed witness. Enrollment and attestation are trusted external inputs; in-process review/replay state and test keys are not a production ceremony. Each artifact's README states its configured trust, mTLS requirements, identity profile and limits. The Rust repository's component-coverage matrix is the implementation handoff; protocol semantics remain language-neutral.

9. Adoption, Ownership, and Deployment Gates

KedgeFlow can be introduced around a selected protected action while existing models, graphs, and orchestration remain in place. OpenFlow, Kubernetes admission, and service-mesh control/data-plane separation offer architectural precedents for adding programmable enforcement to existing systems [4, 71, 72]. They do not predict KedgeFlow's market trajectory or make it a drop-in security boundary. Adoption depends on effective attachment points, credential and writer coverage, target capabilities, operational reliability, and measured integration cost.

9.1. Buyers and Operational Owners

A deployment needs both an organizational sponsor and explicit owners for its architectural roles. Table 11 connects three common interests to acceptance evidence. These are proposed adoption personas, not measured market segments.

Sponsor / Owner	Reason to Adopt	Evidence Needed to Buy or Operate
Security and governance	Restrict confused-deputy authority paths; reconstruct who proposed, admitted and executed each effect.	Verified credential/native-path isolation, subject/workload delegation, trusted approval, retention controls and independently checkable lineage. NIST SP 800-207 can inform the design; this paper does not certify compliance.
Platform engineering and SRE	Coordinate competing changes and preserve recoverable operation state.	Predicate/writer coverage, atomic authority claims, target guard/completion scope, fault/restore tests and explicit unknown/compensation outcomes.
Application and agent developers	Reuse policy, admission, evidence and outcome services across existing workflows.	Working Agent Adapter integration, typed tool descriptors, supported remediation/retry semantics, bounded latency and usable failure handling. Shared services reduce duplicated work but do not eliminate resource-adapter implementation.

Table 11: Proposed buying and operating interests, with implementation evidence for each. NIST zero-trust guidance supplies architectural principles, not an automatic product certification [1].

Appoint one deployment owner accountable for acceptance and incident coordination. Security owns policy and approval trust; platform/SRE operates enforcement, authority state and recovery; the resource owner accepts policy correctness, predicate scope and writer coverage. Application teams own runtime integration. The product owner defines the promised task behavior, interruption/support experience and acceptable operating cost.

Named deputies and on-call escalation prevent stalled or unknown operations from falling between teams. Emergency administrative paths, often called *break-glass* paths, have separately administered authority, explicit scope, auditing and coordination assumptions.

9.2. Start with One Governed Action

A first pilot should use a consequential task whose expected user value justifies the engineering and interruption cost. Use one typed action, one supported runtime attachment, one isolated target, and a target predicate and outcome that the team can verify. Prefer an action whose native conditional update protects the required state. If the policy has no wider shared invariant, a separate capacity reservation and saga are unnecessary; exact admission, conserved grant issuance and atomic claim remain required. Human review is required only by the chosen policy. Hardware assurance is required when that policy demands it.

Nine logical roles do not require nine new services. For example, a team can co-locate the five control roles, place the gateway and resource adapter in a protected execution service, and reuse an existing evidence source. This reduces deployment units, not authority checks. The harness must still lack equivalent target privilege, and issuance and claim must share coordinated durable state. Choose another action if those boundaries cannot be established.

Before pilot approval, the application owner states the protected scope and the platform, security and resource owners accept their work packages. Record workload-specific budgets for integration effort, admission delay, reviewer wait and volume, service availability, unresolved-attempt escalation, and

Work Package	Reuse or Reference Starting Point	Adopter Deliverable
Runtime integration	Existing documented hook or MCP client; supplied signed schemas/fixtures.	Agent Adapter for the selected invocation paths; exact proposal and stable operation identity; supported remediation/status handling.
Policy and trust	Existing IdP, policy engine, telemetry or review service; bounded Cedar/WebAuthn examples.	Trusted subject/workload enrollment, policy owner, provider verification, required review assurance and key lifecycle.
Authority and enforcement	Atomic storage and protected gateway; proxy claim/binding compatibility tests.	Production grant issuer, one issuance/claim domain, independent credential custody, target authorization and bypass denial.
Target and recovery	Existing conditional API and status endpoint; mock adapter and recovery tests illustrate the contract.	Typed Resource Adapter, declared predicate/completion level, authoritative outcome lookup, retention and on-call recovery.

Table 12: A first-slice bill of materials. Reuse reduces new code; the right-hand column remains accountable deployment work. The supplied service and proxy examples have distinct profiles and do not form a turnkey production stack.

operating/support cost. The pilot measures these budgets against the existing workflow; exceeding them triggers scoped redesign or postpones expansion. There is no universal numerical target in this paper.

Measure the baseline and pilot on the same task population. Report added decision delay at the adopter's chosen percentiles separately from human-review queue time and target completion time; record integration effort per runtime and resource, review prompts per governed task, unresolved-attempt age and support time. A denial count alone is not a quality metric: assess incorrectly blocked authorized tasks against labeled policy cases. Expansion requires both the boundary tests and acceptable user/operating cost.

9.3. Adoption Friction and Its Response

Table 13 makes the cost of each boundary visible and assigns a response. A mitigation may reduce engineering or user effort, but cannot waive a contract required by the selected protected action.

Table 13: Adopter friction, an owned mitigation, and evidence for expansion.

Friction / Owner	Practical Response	Acceptance Measure or Stop Condition
Too many components / platform lead	Assign logical roles to existing services before buying new ones; co-locate compatible roles and use one declared authority domain. Keep permission and key ownership explicit.	Named owners and a priced build/buy/retain work plan. Stop if required role ownership or support is missing.
Harness changes and tool churn / application team	Start at one documented attachment; freeze typed inputs and reuse the same operation on remediation. Pin runtime/profile/descriptor versions and run the contract fixtures on upgrade.	Covered-path and substitution tests; measured integration effort. Unsupported paths stay disabled for the protected scope.

Friction / Owner	Practical Response	Acceptance Measure or Stop Condition
Credential migration / security and platform teams	Provision enrollment, broker and protected service custody before cutover; revoke equivalent agent credentials only after the isolated route works. Inventory native tools, sockets and other writers.	Direct-bypass, revocation and secret-access tests. A working proxy without custody/coverage is still a pilot.
Legacy target limitations / resource owner	Govern one typed API action; reuse a native conditional guard where it covers the predicate. Declare B/C exposure and recovery explicitly.	State-change and outcome-query tests. Block a protected action if its required predicate cannot be enforced.
Approval fatigue / policy and product owners	Request review only for policy-required obligations; reuse still-valid static checks through non-authorizing leases. Route one canonical pending proposal to a reviewer and resume it asynchronously.	Review volume/wait, repeated prompts and authorized-task abandonment within adopter-set budgets; no blanket or stale approval reuse.
Latency and dependency outages / SRE	Overlap permitted planning with admission; reuse authenticated broker connections and budget exchange plus final dispatch checks within signed validity. Measure the complete route, including optional grant-store lookup and header limits.	Admission/exchange tail latency, evidence age, remaining dispatch margin and fault tests. Expiry never permits lifetime extension or fresh authority. Required admission/claim failure blocks new protected dispatch.
Ambiguous failures and support load / SRE and resource owner	Retain durable operation/attempt IDs, correlate authoritative target status, expose pending/unknown state and set bounded polling with an owned escalation deadline. Qualify recovery before admitting legacy actions.	Lost-response, late-effect and escalation tests; unresolved age and operator time. Deadline or incident closure cannot clear consumption or justify an unsafe inverse.
Upgrade cost and lock-in / platform and procurement owners	Require versioned descriptors, exportable contracts/lineage and an explicit key/state transition plan. Qualify replacements against the same scope and fault cases.	Upgrade and migration rehearsal; no transparent conversion of signed records, replay of consumed roots or untested portability claim.

9.4. A Four-Stage Rollout with Explicit Gates

Figure 5 outlines a candidate rollout. The stages describe growing verified coverage, not permission to weaken the execution contract on an already protected action. Single-use claim is required as soon as grants authorize execution; shared-invariant reservations are added when the declared predicate requires them.

1. Observe. An Agent Adapter or existing tracing middleware records selected invocations and evaluates candidate policies without changing execution authority. Captured arguments can support a provisional proposal; missing identity, evidence or outcome data must remain missing. A log does not create a verified witness or target commit receipt. Observation can expose credentials or add overhead, so redaction, access controls, retention and performance are part of the pilot. This stage establishes an inventory and policy hypotheses, with no claim of prevented mutations.

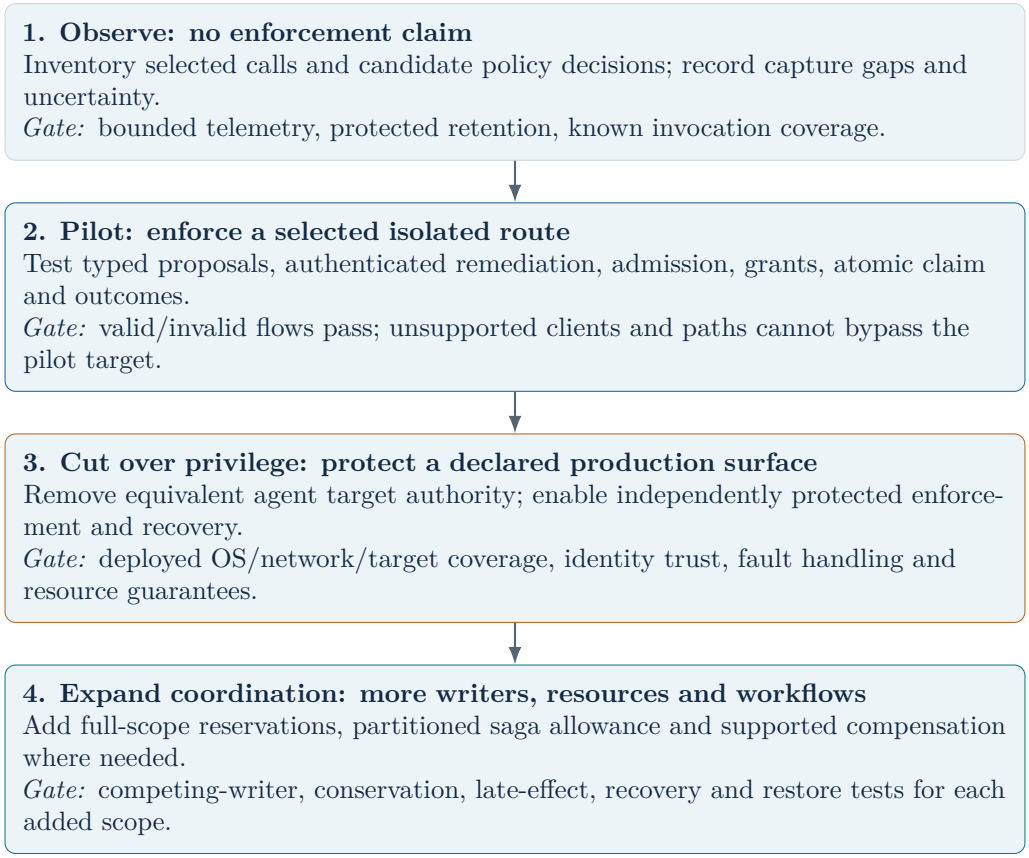


Figure 5: A proposed rollout, with the assurance boundary stated at every stage. Shadow observation is not protected execution, and expanding a deployment cannot postpone the claim/credential rules of an existing protected route.

2. Pilot a governed route. Use an isolated or nonproduction target and an explicitly supported runtime adapter. Exercise the existing `NEEDS_EVIDENCE` contract, denied/unknown results, exact parameter and identity binding, grant issuance/claim, and the adapter's declared outcome query. The pilot must restrict its target authority sufficiently to test enforcement. A proxy on one call path can validate that route, but remaining native credentials elsewhere prevent a wider protection claim. Authorized observations use their own confidentiality and scope rules.

3. Cut over protected privilege. Revoke or restrict equivalent agent credentials for the chosen production resource/action surface; provision independently protected gateway/broker custody and target authorization. Verify alternate shell, file, socket, SDK and delegated paths, user/workload trust, policy/evidence freshness, credential downscoping and outage behavior. Preserve separately administered break-glass procedures. A cutover is accepted only when bypass and resource-specific fault tests pass; fallback to an ambient credential would undo the boundary.

4. Expand coordination and recovery. Add new resources and participating writers to declared conflict domains, and coordinate any shared invariant before expanding autonomous operation. Two grants in the same conflict domain need not be mutually exclusive if the invariant permits both; competing removals of the last spare capacity must be. Saga orchestration allocates conserved child allowances and runs registered, freshly admitted compensations only when known outcomes and current

guards justify them. It is conditional recovery, not automatic atomic rollback. Unknown forward effects first require reconciliation and a late-effect barrier.

9.5. Deployment Packaging and Guarantee Limits

The architecture permits several packaging choices. A local developer distribution can combine the controller roles, a protected gateway, descriptors and compatibility vectors; single-host storage still needs the same atomic issuance/claim and recovery semantics. An enterprise service can centralize policy administration, review trust and retained lineage while gateways remain close to resources. A provider integration can supply workload enrollment, target-specific credentials and native guards. These are design options, not announced products, tested marketplace installers or revenue forecasts. Multi-region availability additionally needs explicit authority ownership, quorum/partition and restore contracts; deployment geography alone does not supply them.

CTDs let teams govern a selected consequential surface while retaining permitted workspace operations. The friction budgets in Section 9.2 are measured on the actual workload; neither locality nor static leases establish a cost or latency advantage. Leases reuse still-valid admission work but do not authorize execution or replace final freshness checks.

Legacy resources can use declared Level B or C contracts when the selected policy accepts their check-to-effect exposure or delayed uncertainty. If the required protected-action predicate cannot be enforced, the action must block until an adequate target mechanism exists; unenforced policy evaluation belongs only to the explicitly labeled observation stage. Protocol neutrality requires independently working adapters and common fixtures; MCP/A2A transport support alone is insufficient.

9.6. Workflow Interruptions and Safe Next Actions

The Agent Adapter and product UI expose the existing result and lifecycle semantics faithfully. Table 14 is integration guidance, not a new wire error or state profile. Each protected task shows the exact action, a policy reason safe to disclose, its proposal/attempt reference, current status, responsible reviewer/operator and next supported action. Deduplicate review prompts by stable proposal/operation identity. Editing a reviewed action creates a new proposal and admission; a terminal confirmation cannot substitute for a required hardware-review obligation.

Table 14: Existing results mapped to workflow handling and user communication.

Result or Lifecycle State	Workflow Response	User Status / Accountable Owner
`NEEDS_EVIDENCE`	Pause the unchanged proposal; validate the trusted remediation path and obtain only the required evidence. Resume exact admission rather than replanning silently.	Awaiting named evidence/review, with expiry and a status reference. Application/product owner and review provider.
`DENIED`; identity/grant mismatch	Stop protected dispatch. Route policy changes through authorized administration; investigate identity/binding errors. Do not repeatedly ask another reviewer for the same refused action.	Denied or binding error, safe reason and escalation path. Policy/security owner.

Result or Lifecycle State	Workflow Response	User Status / Accountable Owner
Stale policy/evidence; `STATE_GUARD_VIOLATION`	Refresh and revalidate through admission. Inspect existing root/attempt state before issuing replacement authority; changed bound context or parameters require the appropriate new proposal.	Action needs revalidation, not completed. Admission/resource owner.
`ROOT_UNAVAILABLE`; `NONCE_CONSUMED`	Query the existing authority/attempt. A new RPC ID, nonce or automatic tool retry cannot create an additional allowance.	Existing operation or authority issue; status lookup and operator escalation. Platform owner.
`dispatched` / `pending`	Query the correlated target operation under its completion contract; do not redispatch to obtain a faster response.	Accepted or executing, with no premature completion claim. Resource adapter/SRE.
`unknown`; `OUTCOME_UNKNOWN`	Retain consumption and any reservation; reconcile authoritative evidence and escalate at the declared deadline. Another action does not clear uncertainty.	Outcome unknown, last confirmed fact and incident reference. Resource owner/SRE.
`committed`; `failed` / `compensation_failed`	Report the receipt's actual effect scope. Failure does not imply zero effect or refund; any supported compensation needs fresh admission and known preconditions.	Confirmed effect or failure with possible partial effect. Resource/product owner.
`not_dispatched`; `fenced_no_effect`	Preserve the verified refusal/resolution; `not_dispatched` alone does not restore an allowance. Authoritative no-effect evidence plus the required late-effect barrier can support restoration under issuer conservation rules.	No dispatch or fenced no effect, with supporting evidence. Gateway/resource owner.

Before claim, UI cancellation may withdraw a proposal or request atomic retirement of unclaimed authority under the authority contract. For claimed or dispatched work, cancellation stops future work only under the declared target contract: it does not prove no effect, release a reservation or undo a commit. Without authoritative resolution and any required late-effect barrier, the attempt remains pending or unknown. The product owner explains this distinction before launch.

9.7. Operate, Upgrade, and Exit Without Reopening Authority

During an outage, permitted planning and workspace work can continue within their policy. New protected dispatch stops when required admission, claim or credential authority is unavailable. An Outcome Service outage may buffer already supported records in durable outboxes under the declared contract; if a required transition record cannot be durably retained, that transition must stop. Disconnected operation requires a supported local authority/target profile preserving custody, freshness, atomic claim and recovery. Cached policy or admission alone is insufficient.

The deployment owner rehearses cutover, abort, upgrade, restore and supplier exit as one authority-preserving transition:

1. Quiesce new issuance and dispatch; establish the resource's fencing or late-effect barrier where required. Inventory every granted, claimed, pending and unknown attempt. Stopping the controller does not cancel target work.

2. Reconcile or quarantine in-flight operations. Retain consumed roots, generations, reservations, replay state, dispatch intents and outboxes; a stale restore or a fresh empty ledger cannot make old authority reusable.
3. Preserve original signed records, trusted key/version histories and descriptor/policy bindings. Validate the new runtime/profile and key transition; incompatible signed records are not silently rewritten or re-signed as proof of the old history.
4. Run the same scope-specific refusal, bypass, concurrency, lost-response and restore cases before resuming. Separately authorized emergency operators remain declared writers. Returning equivalent ambient credentials to an agent changes or ends the protected scope and requires explicit owner acceptance.

The acceptance record names dependency patching/provenance, key rotation, outage backlog, recovery deadlines and on-call ownership. A failed pilot can disable the protected feature while retaining permitted work and a separately governed operator procedure. This makes postponement practical without treating an unsafe fallback as a successful migration.

9.8. Buyer Control and Supplier Evidence Record

All buyers need a deployment-specific evidence record. Government and sovereign procurement teams additionally use it to state who controls the deployment and its dependencies; local hosting alone does not establish that control. Name the operators and administrators of policy, grant signing, workload enrollment, Evidence Providers, credential brokers, optional grant stores/resolvers, gateways, authority state and retained outcomes. Record which external model, judge, identity, signing or support service can observe, alter or disable the chosen operation, and whether the buyer can operate the required path independently. Declare provider independence and conflicts of interest rather than inferring them from multiple signatures.

Inventory the sensitive canonical proposals, identity, evidence, review and receipt records: access, storage/transfer location, retention, permitted disclosure and export. Redacted views retain links to the protected originals; altering a signed payload breaks its signature and digest bindings. Specify who may verify the retained chain, what records verification requires, and how originals/key histories survive supplier exit.

The resource/public-service owner accepts policy correctness and the residual limits of B/C adapters, external writers and uncertain effects. The buyer selects any legal, national-assurance or accreditation requirements separately and obtains evidence for its chosen profile. No sovereign-operation, production-resilience or supplier-portability result is established by this whitepaper.

9.9. Application-Team Onboarding Record

Before accepting a protected deployment, retain one reviewable record containing:

1. Resource/action scope, CTDs, all relevant writers and credential/socket/native-tool routes, including excluded or separately administered paths.
2. One accountable deployment owner and named component/interface, product-support and incident owners; trust enrollment, secret-store access policy, active runtime/profile/schema/descriptor versions, policy epoch, gateway audience, resource incarnation and authority domain.

Acceptance Topic	Evidence Supplied Here	Evidence Required from Deployment
Contract exchange	Closed schemas, signed vectors and independent canonicalization/signature checks.	Frozen versions and independently implemented peers exchanging the selected profile.
Protected boundary and review	Proxy identity/claim tests; mock guards; software-key WebAuthn mechanics.	Actual enrollment, hardware assurance when required, native bypass/custody and selected-target guard/outcome tests.
Recovery and operation	Bounded claim races and mock lost-response reconciliation; architecture acceptance cases.	Real fault/restore, key transition, emergency operation, data controls, support and safe supplier-exit rehearsal.
Adoption value	Proposed work packages, friction responses and measurement method.	Workload-specific user/operating cost, accepted guarantee limits and signed-off expansion decision.

Table 15: Procurement evidence split. Section 10.4 states the scope of supplied checks; Section 10 supplies deployment tests. This is an acceptance input, not a certification or jurisdiction-specific accreditation claim.

3. Required obligation classes, review assurance, authority/claim domain, target predicates, completion modes and compensation limits.
4. Executed missing-grant, forged/expired evidence, identity/parameter substitution and direct-bypass cases, with no target dispatch on refusal.
5. Competing-writer, concurrent-redemption, lost-response, late-effect and restore tests; consumed unknown authority cannot reopen through a new request ID.
6. Measured operating/user cost, data/dependency control and the rehearsed outage/upgrade/exit procedure, with an explicit list of partial or unverified contracts. Section 10 supplies the technical cases and Section 9.8 the buyer evidence split.

This record gives buyers a concrete completeness assessment and gives implementers an integration backlog. Each new tool is assigned by the obligation, owned authority and failure contract it supplies, preserving the component vocabulary as the ecosystem changes.

10. Acceptance Criteria, Evidence, and Release Gates

KedgeFlow's architecture supplies a checklist for a complete control plane. Passing that checklist requires implementation evidence for a declared resource/action surface. The matrix below states deployment acceptance requirements. The distribution also reports bounded executable artifact checks separately; passing those checks does not establish the full deployment matrix or cross-vendor interoperability.

10.1. Candidate Conformance and Acceptance Matrix

Each implementation identifies its protected resources, action schemas, trust boundaries, adapter guarantees, completion modes, and exclusions. Tests exercise the deployed credential and resource boundary, including alternate tool, shell, delegated, and direct API paths. A framework callback passing a unit test is insufficient evidence of complete mediation.

Test Category	Scenario	Required Evidence and Behavior
Coverage	Agent attempts an in-scope mutation through a direct API, shell, subagent, or alternate credential.	Resource-side controls reject the bypass. Coverage inventory names every mediated path and any exclusion.
Local Privilege	Read gateway secrets/configuration, attach to its process, use SSH/container sockets, or trigger privileged automation through workspace edits.	OS, service and target controls deny equivalent mutation authority. Credential storage alone is insufficient; workstation containment is tested independently.
Local Routing	Use built-in shell/file tools, alternate MCP/SDK paths, subagents, substituted gateway endpoints, or an invalid Host/Origin.	Protected effects remain mediated. Endpoint rejection precedes authority use; accepted local clients still authenticate. MCP configuration alone cannot establish coverage.
Local Execution	Change an approved script/hook, symlink target, environment, working directory, or repository ref after review; disclose secrets via a permitted read.	The adapter binds complete effect inputs and enforces the declared guard. Confidentiality/egress policy constrains observations; no general shell-query exemption.
Message Integrity	JSON field order changes; duplicate keys, forged signatures, or conflicting message-ID reuse are submitted.	Canonical reordering preserves the digest; invalid or conflicting messages are rejected before an authority claim.
Dual Identity	Substitute subject issuer/ID, workload SPIFFE ID, DPoP key, or delegation claims; replay a proof for another endpoint.	Reject before claim. Deployment evidence includes real token/proof validation and independently authenticated workload context, not trusted test headers.
Credential Minting	Broker returns an excessive scope, wrong target audience, expired token, or exchange failure after claim.	No target dispatch; retain the claimed attempt. Target credential is confined to enforcement and inbound MCP/DPoP/grant credentials are not passed through.
Exchange Latency	Provider/assertion/response or final ledger delay consumes remaining validity; credential receipt time overestimates lifetime.	Bounded exchange, conservative lifetime and final validity check prevent expired initiation. Retain consumption on refusal; cancellation after possible send remains uncertain. Measure provider-tail and dispatch-margin budgets.
Grant References	Large envelope, mixed/duplicate carriage, substituted digest, wrong identity, unavailable/redirecting resolver, oversized response or replay through another carriage.	Negotiated pinned authenticated lookup; original envelope verification and same claim. Failure before claim where retrieval/binding fails; no caller-token forwarding or URI fetch. Verify the whole intermediary route separately.
Hardware Review	Approve another proposal/policy/rendering digest; use a non-UV, replayed, expired, or unenrolled hardware assertion.	Reject as unverified. Hardware enrollment, trusted rendering and WebAuthn challenge validation are tested separately from evidence-provider signature verification.
CTD Binding	Tool schema, consequence tier, fence parameter, guard primitive, or compensation descriptor changes after admission.	Reject mismatched digest or unsupported adapter contract; server annotations cannot silently lower the guarantee.
Exact Scope	Caller, audience, resource incarnation, schema, or parameters differ from the grant.	No claim or dispatch. The signed diagnostic identifies the failed binding.

Test Category	Scenario	Required Evidence and Behavior (continued)
Duplicate Issuance	Competing issuer replicas receive the same admission through different request IDs.	One persisted issuance and execution allowance; a new request ID cannot multiply authority.
Claim and Transfer	Gateway replicas redeem concurrently while an issuer replaces or delegates the grant.	Atomic root-generation validation, claim, and retirement admit at most one dispatching attempt. Claimed or uncertain attempts block transfer until safely resolved and fenced.
Recovery	Issuer or gateway crashes around claim/dispatch; a stale backup or partitioned replica restarts.	Consumed authority remains consumed. Recovery preserves attempt identity and cannot reopen a root from stale state.
State and Conflict	Relevant resource state changes; two proposals compete for the last spare capacity.	Level A rejects a false declared guard atomically. Shared-invariant protection additionally requires a coordinated reservation covering the complete predicate and competing writer set.
Saga Conservation	Allocate parallel/nested children, then fail a downstream step after confirmed predecessors.	Atomic allocation prevents budget duplication. Compensation uses its reserved allowance and fresh grants; unknown effects block unsafe inverse execution. Partial compensation remains explicit.
MCP / A2A	Invoke a protected batch, omit extension negotiation, alter parameters on remediation retry, or delegate outside the advertised parent allowance.	Reject unsupported carriage or scope. Transport task/request IDs do not create execution authority. Independently implemented SDKs are required before claiming portable interoperability.
Admission Lease	Revoke membership, activate policy, change descriptor, or expire a fast witness while holding a static lease.	Lease supplies no execution authority. Final admission revalidates assumptions and JIT obligations; commit guard remains required.
Evidence Freshness	A witness expires while the resource version stays unchanged.	No affirmative admission or execution past the witness lifetime at the check point required by the grant's completion mode.
Policy and Revocation	Cached policy rolls back or ages past its bound; revocation races execution.	Persisted version/epoch checks prevent rollback. Dispatch or commit obeys the declared propagation and completion contract; partition behavior is explicit.
Ambiguous Outcome	Target may commit, but the response or receipt is lost.	The attempt remains consumed and unknown until authoritative reconciliation. Status lookup does not redispatch; silence is never treated as proof of no effect.
Legacy Escalation	Level C target lacks authoritative query/barrier; reconciliation reaches its declared deadline.	Open the owned incident and preserve unresolved state/consumption. No timer-based refund or unsafe inverse; separately governed mitigation cannot clear original uncertainty.
Outcome Outage	Outcome Service is unavailable; reports are duplicated, delayed, or reordered.	Durable outboxes retain decision/dispatch lineage. Append is idempotent; contradictory receipts remain visible.

Test Category	Scenario	Required Evidence and Behavior (continued)
Independent Trace	A separate verifier reconstructs a committed or unresolved attempt.	It verifies proposal, evidence, policy, certificate, grant, claim, and outcome links, reporting missing links and limits of resource attestation.

Table 16: Proposed KedgeFlow acceptance cases. Each result must be demonstrated for the implementation's declared surface.

10.2. Comparison with a Hardened PDP/PEP Baseline

KedgeFlow combines existing authorization and systems mechanisms behind explicit interfaces. Its architectural value cannot be established by comparing it with an agent that simply holds an administrator token. A suitable baseline uses a capable Policy Decision Point (PDP), such as OPA, and a resource-side Policy Enforcement Point (PEP), with equivalent credential isolation, evidence feeds, exact-action capabilities, coordinated claims, guards, idempotency, and reconciliation [1, 21]. If that assembly implements the KedgeFlow contracts, it is a possible KedgeFlow implementation. Functional equivalence is a valid result.

Comparative evaluation should measure:

- **Boundary enforcement:** unauthorized dispatches, stale or false declared guards, replay, conflicting reservations, and partition/recovery behavior under matched fault injection.
- **Decision quality:** false approvals and false denials under explicitly labeled evidence faults and provider correlations. Signatures and multiple providers do not establish truth or independence by themselves.
- **Operating cost:** latency, throughput, evidence-acquisition cost, and unresolved-attempt duration, with admission, issuance, claim, dispatch, and reconciliation measured separately.
- **Integration and portability:** engineering work to add a runtime, resource adapter, evidence provider, or new policy; successful exchange between independently implemented interfaces.
- **Audit reconstruction:** the completeness and cost of independently verifying authorization lineage and resource outcomes, including explicit unknowns.

Report attempted, dispatched, committed, rejected, and unresolved populations separately. For each observed effect, record independent labels for authorization-path validity, actual satisfaction of declared obligations/guards, and business outcome. A valid grant based on a deceptive witness is different from a forged grant; an undesirable outcome under an incomplete policy is different from a bypass. Keep unknown labels explicit. Where exclusive failure totals are needed, use the precedence: invalid authority, then violated explicit obligation/guard, then undesirable outcome despite a satisfied contract.

10.3. Evidence Maturity and Release Gates

This revision supplies an architectural proposal, candidate machine-readable profiles, signed fixtures, and bounded reference implementations. The accompanying verification report records the commands and results actually executed. It reports no completed cross-vendor interoperability exercise, production conformance result, hardware deployment certification, or comparative efficacy benchmark. A local

three-member etcd cluster exercises real Raft transactions on one host; it does not measure geographically distributed performance or prove partition/recovery behavior beyond the named tests.

A reference implementation or protocol release should supply:

1. **A frozen interface profile:** machine-readable schemas, caller-role permissions, canonical signing bytes, key/version rules, and positive and negative signature vectors. Example JSON alone is insufficient.
2. **A verified enforcement surface:** resource/action coverage, credential separation, adapter predicate scope, commit point, completion mode, and bypass tests against the deployed target.
3. **Fault and concurrency evidence:** reproducible issuance, transfer, reservation, claim, recovery, expiry, revocation, and ambiguous-outcome tests with raw signed traces.
4. **Independent reconstruction and exchange:** a trace verifier and independently implemented peers exchanging the same fixtures before claiming interoperability.
5. **Reproducible evaluation:** pinned tool versions, policies, workloads, fault corpus, resource descriptors, baseline configuration, and raw results before claiming efficacy or operating advantages.

10.4. Executed Artifact Checks in v0.9

The evidence below is attributed to its artifact. The Rust repository is the primary implementation baseline; the earlier Go runs remain historical evidence for the supplementary prototypes. Test counts, race-detector results and target behavior are not transferred from one implementation to another.

Shared schema and compatibility vectors. The delivered schema suite validates all thirteen signed fixture envelopes, cross-record bindings, complete paper-schema equality, CTD YAML equivalence, and the declared integer-only canonicalization domain. Twenty-one profile tests exercise field closure, required security fields, tampering, identity binding, review substitution, hardware-emulator exclusion, unsafe budget/cycle examples, failure receipts, and no-effect evidence. An independent Node verifier checks Ed25519 signatures, canonical digest bytes and review challenges. These are compatibility vectors and negative contract tests, not live authorizations.

Primary Rust gateway: recorded checks of 5 October 2026. At baseline `0a06d00`, fifty-two Rust tests pass with the real three-member etcd v3.6.5 cluster and official MCP SDK required; twenty-four Python compatibility tests also pass. Rust formatting and clippy, Python lint/type checks, signature/semantic fixture validation, and byte-identical schema regeneration pass. The recorded compiler is Rust 1.98.0; the locked dependency minimum is 1.88, which was not separately exercised.

The Rust checks cover claim contention across roots and member endpoints, replacement and policy/ownership races, authenticated TLS transport, access-token/DPoP negatives, credential timing, grant references, and a fenced mock target with confirmed, stale-guard and lost-response outcomes. The official MCP client exercises discovery, remediation, signed-fixture retry, terminal result and replay refusal. Its authentication ports are explicitly synthetic; cryptographic identity and mTLS are checked separately. These runs establish the named local mechanics, not production target coverage, distributed availability, hardware review, a full admission-to-execution deployment or saga recovery. The exact checks and raw results are retained in the Rust repository's `docs/RUST_VERIFICATION.md` and `results/verification/`.

Historical Go proxy and service experiments. The Go proxy's integration tests start three actual embedded etcd v3.6.5 servers and communicate through the etcd client. The following reported Go matrix uses 1, 8, and 32 roots with 32 or 128 contenders per root; it is not a Rust throughput result:

Roots	Contenders / Root	Attempts	Successful Claims
1	32	32	1
1	128	128	1
8	32	256	8
8	128	1,024	8
32	32	1,024	32
32	128	4,096	32
Total		6,560	82

Each root has exactly one winner; 6,478 contenders are refused with zero transport errors in that run. Another test races claim against generation replacement one hundred times, verifies the exclusive winner, and rejects stale generations and fences. The cluster runs on one macOS arm64 host with loopback networking and ten logical CPUs, using Go 1.26.1. Raw logs record elapsed time and attempted claims per second. This workload is dominated by refused redemptions; it does not estimate independent successful-mutation throughput or wide-area latency. Prefix grouping is not evaluated as Raft sharding.

The historical Go proxy also passes its pinned official SDK integration: `@modelcontextprotocol/client` v2.3.0 with Node v25.2.1 and protocol version `2026-07-28`. The unmodified SDK client and Streamable HTTP transport perform discovery, tool listing, missing-grant `NEEDS_EVIDENCE`, resubmission with a signed fixture grant, and `NONCE_CONSUMED` replay rejection. Exactly one mock effect and one target credential are produced. Documented client options negotiate the extension; a custom fetch function adds the grant header without rewriting RPC bodies.

This test uses an explicitly injected loopback identity and a preissued grant. Cryptographic authentication is tested separately; the SDK check does not demonstrate automatic admission by an unmodified agent harness or a complete service-to-proxy deployment.

The separate capacity-admission service uses the pinned `cedar-policy/cedar-go` v1.8.0 engine. Its HTTP test exercises signed proposal validation, a WebAuthn challenge, a cryptographically verified assertion, witness ingestion, Cedar evaluation and a schema-valid signed admission certificate. Missing, stale and unknown capacity evidence defer; violated capacity denies; satisfied obligations admit only if Cedar permits. A Cedar evaluation diagnostic denies even when another policy permits.

Assertion tests use software-generated Ed25519 and ES256 keys with the actual WebAuthn signature input. They reject substituted challenge/proposal/template/policy, wrong origin/RP, missing user verification, unknown credential and invalid signature. An eight-contender replay test permits one assertion consumption. Hardware-required configuration rejects the explicitly test-only enrollment. These checks validate assertion mechanics and policy separation; they do not report physical hardware registration, attestation-chain validation or a browser/human ceremony. The admission and proxy examples use distinct bounded profiles and do not demonstrate a complete interoperable deployment.

Credential-timing tests exercise delayed exchange and assertions, conservative credential lifetime, expiry after exchange or during the final ledger check, and retained consumption on refusal.

Eight grant-reference transport test groups with fifty-nine subcases exercise pinned mutual TLS, requester scope, no caller-token forwarding, negotiation, ambiguous carriage, redirect/size/deadline refusal, binding failures before claim and shared inline/reference replay. A 14,272-byte signed mock grant, requiring a 19,030-byte inline value, is refused inline and accepted through the authenticated reference route with one

claim, credential and mock effect. These tests do not add multi-predicate, Level B/C, or saga support. The full race-detector suite passes with the official SDK required. Store-side publication/authorization, real intermediary header tests and enterprise provider latency remain deployment work.

The local CLI additionally validates its configured HTTP endpoint boundary: unexpected Host/path and supplied Origin are refused before the proxy handler. These focused checks do not establish workstation isolation or full MCP transport conformance. Native shell/file interception, keychain custody and local Git/filesystem enforcement remain undeployed. The complete reproduction commands, subsequent checks, raw logs, and remaining gates are retained in `docs/V0_9_VERIFICATION.md` and the artifact READMEs. A Docker build recipe is supplied for the proxy; a container launch is not claimed because the execution environment has no reachable Docker engine. Complete deployment acceptance still requires the selected external target and trust infrastructure, additional independent Agent Adapters and clients, fault/restore tests and resource-specific writer coverage.

10.5. Scope of the Architectural Claim

Under uncompromised trusted components and keys, verified complete mediation, and the specified atomic authority state, an implementation can enforce grant-bound dispatch for its declared surface. Level A additionally protects only the predicates atomically enforced by the target. A local object version does not establish a fleet-wide capacity invariant, and an admission-time telemetry check does not establish commit-time freshness. Commit-bound claims require an enforceable check at the actual commit point.

KedgeFlow does not establish semantic correctness, complete dependency discovery, truthful evidence, exactly-once external effects, automatic workflow rollback, or immediate revocation during a partition. These require further domain assumptions or resource mechanisms. The contribution is an explicit blueprint for identifying those requirements and exposing gaps rather than hiding them behind an agent framework, a guardrail score, or a gateway product name.

A. Conditional Enforcement Invariants

The following contracts state what an implementation must preserve. They depend on the declared resource/action surface, uncompromised trusted services and keys, authenticated identities, complete mediation, durable atomic authority state, and the resource adapter's actual enforcement capabilities. They are conditional mechanism claims, not proofs of business correctness or reported implementation results.

Proposition 1 (Grant-Bound Dispatch on the Declared Surface)**.** For every protected mutation dispatched within a verified mediated surface, the Execution Gateway has authenticated the caller and validated an authentic grant matching the exact proposal, audience, resource incarnation, parameters, policy conditions, and required validity bounds. The gateway has atomically claimed that grant's active root generation before dispatching through the Resource Adapter.

This follows only if all relevant mutation paths reach the gateway and its trusted adapter. A runtime hook or network proxy with a remaining direct credential path does not satisfy the premise. The claim concerns the verified authorization path; a correctly signed witness can still contain false information.

Proposition 2 (Level A Guard Preservation)**.** For a Level A adapter, let g_s denote the declared predicate that the target evaluates atomically with mutation. A successful commit implies $g_s(s_{\text{precommit}}) = \textsc{True}$ at the atomic check-and-commit point. If that predicate is false, the guarded mutation produces no effect.

Guard satisfaction is necessary, not sufficient, for commit: resource failures, independent constraints, expiry, or denial can still prevent execution. Changes to state outside the fields covered by the predicate need not invalidate the action. A compare-and-swap on one object protects that object's declared version condition; it does not atomically validate telemetry, other objects, or a global dependency graph. Any wider invariant needs a transaction, enforceable reservation, or equivalent resource mechanism covering its full read/write set.

Proposition 3 (Conservation of an Execution Allowance)**.** For each executable leaf root R, issuance, replacement, transfer delegation, and gateway claim share a coordinated atomic authority record. A composite saga root partitions a conserved allocation into independently claimable leaf roots and is not itself redeemable. At most one current grant is redeemable, and at most one dispatching attempt can claim a root generation. Replacement or delegation cannot make a consumed generation reusable or multiply the root's execution budget.

The claim transaction checks root identity, active grant, generation, retirement status, nonce, and any required reservation together. Retirement competes in the same serialization order. If claim wins, an unknown attempt remains consumed and blocks transfer. A confirmed effect spends the root's budget. Only an authoritative no-effect result plus fencing of late dispatch can permit fresh admission and a new generation within that budget; reconciliation alone cannot reopen it. A fresh nonce for an old generation, separate issuer/gateway caches, or a stale database restore cannot reset consumption. This contract limits authorized attempts; it does not imply exactly-once effects in an external API without its own idempotency or transactional support.

Shared-invariant coordination. Reservations protect only an explicitly modeled invariant and accounted-for competing writers. Two removals of the last spare capacity must compete for the same authoritative allocation, and target execution must honor the reservation's scope and fencing. Writers outside this coordination domain, unmodeled dependencies, or lost fencing invalidate a global preservation claim. Observing *no other operation in flight* is insufficient to reserve capacity.

Outcome evidence. A signed gateway receipt authenticates what the gateway reported. It proves a target effect only under a trustworthy adapter observation or an independently verifiable target receipt. An append-only hash chain can expose alteration against retained checkpoints; it does not alone guarantee immutable storage, complete logging, truthful receipts, or legal non-repudiation. Reconciliation preserves unknown outcomes until adequate resource evidence is available.

B. Durable State and Atomicity Boundaries

These are logical records and consistency requirements, not mandatory database products, key prefixes, or separate processes. State ownership is part of the architectural boundary. Observations and catalogs remain distinct from execution authority even when stored in the same database.

Role / Store	Owned Records	Consistency Contract
Policy Authority / policy store	Versioned policies, activation/compatibility rules, revocations, ownership epochs, distribution checkpoints.	Signed monotonic versions; persisted rollback protection and declared freshness/propagation bounds.
Resource Registry / catalog and observation store	Identities, action schemas, resource incarnations, coverage descriptors, provenance-tagged observations and dependencies.	Versioned descriptors and explicit unknown/stale values. Registry observations are not atomic target snapshots or authoritative reservations.
Admission Controller / admission store	Frozen proposals, obligation sets, witness references, discharge records, certificates.	Immutable digest-linked records; certificate binds the admitted constraints. Unknown or expired required evidence cannot yield affirmative admission.
Grant Authority and Execution Gateway / coordinated authority ledger	Issuance identity, roots, active grants/generations, reservations, claims, retirement/fencing state, attempts, dispatch intents.	Root activation, claim, and transfer have one atomic serialization contract. Claim validates active generation and consumes the allowance; reservations are atomically allocated where required.
Execution Gateway / local durable records	Attempt status, target idempotency/operation IDs, policy high-water marks (highest accepted versions), pending receipts.	Durable dispatch intent before external invocation; replica claims use the coordinated authority ledger. Recovery preserves consumed and unknown attempts; retained policy versions prevent rollback.
Outcome Service / outcome store	Signed receipts, retained lineage, append/deduplication keys, unresolved attempts, reconciliation evidence.	Append and receipt linkage are idempotent and tamper-evident against retained checkpoints. Outcomes preserve contradictions and explicit uncertainty.

Table 17: Logical storage ownership and the required authority atomicity boundary.

Issuance and claim transactions also persist the records needed to publish their audit events through durable outboxes. If a required record cannot be durably retained, that transition must not proceed. Target commit remains a separate boundary unless the target participates in the same transaction or implements an equivalent conditional/idempotent operation. The ledger cannot manufacture atomicity for an external API.

C. Identity, Review, and Saga Invariants

Proposition 4 (Dual-Identity Confinement)**.** For a protected dispatch, the validated subject, workload, proof-of-possession key, and delegation digest equal the grant's bound context. Target credential issuance follows a successful claim and cannot enlarge the intersection of the subject's rights, workload policy, registered target contract, and exact grant scope.

This depends on trusted token/workload verification and a broker that enforces the downscoping contract. A JSON field named `workload_id` is not authentication; a static target credential at the agent bypasses the premise. Token exchange and target commit are distinct failure points.

Proposition 5 (Review Binding)**.** A hardware-review obligation is satisfied only by a trusted provider's verified WebAuthn assertion whose challenge commits to the exact proposal, active policy, immutable template, rendered transaction, reviewer, verdict, and validity record.

This assumes an enrolled credential satisfying hardware policy and an honest review renderer/client. Cryptographic challenge binding prevents substituting a different admitted payload; it does not independently prove that a person understood the display or that an untrusted display was honest.

Proposition 6 (Hierarchical Budget Conservation)**.** A composite root's initial typed allowance is the sum of its exclusive free balance and the available, held, and spent balances of disjoint child subtrees. Atomic partition delegation and retirement preserve this equality. Each executable leaf separately preserves single-use claim and generation fencing.

A child subtree includes descendants exactly once. Compensation allocations are included in the original allowance. Known effects spend their allowance; unknown effects hold it. Refunds require the target's authoritative no-effect and fencing contract. Compensation consumes a fresh allocation and cannot establish global atomic rollback.

D. Complete Machine-Readable Contracts

The complete schema and signed compatibility vectors accompany this paper under `acp/spec/`; matching contract files are also present in the sibling `openkedge/code/kedgeflow` package. The printed appendix indexes those files instead of repeating the full JSON schema. These artifacts support schema, signature and cross-record checks, not a formal verification of production enforcement. In the sibling package:

- `spec/kedgeflow.schema.json`: The proposed closed Draft 2020-12 schema defining Proposal q, Admission Certificate C_q, Execution Grant G, Consequential Tool Descriptor (CTD v1.0), Attempt Receipt, and all supporting identity, evidence, lease, saga, delegation, remediation, and A2A structures. Unknown protocol fields fail validation; application-data fields obey their registered schemas.
- `fixtures/`: A dedicated folder containing the listings of canonical offline test fixtures, signed Ed25519 envelopes, demonstration keys, and JWT vectors.
- `spec/validate.py`: The offline contract-validation harness that checks schema conformity, signature validity, RFC 8785 canonical serialization, and cross-record semantic bindings.

D.1. Normative Contract Envelopes

Each envelope carries a typed record for a named KedgeFlow lifecycle exchange, as summarized in Table 18.

D.2. Canonical Serialization, Hashing, and Signatures

All signed records conform to profile `KF-JSON-0.2`. Let JCS(x) be RFC 8785 canonical UTF-8 JSON, NUL the single byte `0x00`, $D(k)$ the domain-separator UTF-8 bytes `KedgeFlow/KF-JSON-0.2/`k, and E a signed envelope. The suffix k names the record kind or value purpose; sk is the signing key; $\|$ concatenates bytes; and $E \setminus \{\text{signature}\}$ removes the envelope's signature field.

Envelope Kind	Complete Payload Type	Required Issuing Role	Bound Interface / Purpose
`proposal`	`Proposal`	Agent Adapter	Runtime Admission (q formulation)
`certificate`	`AdmissionCertificate`	Admission Controller	Runtime Admission (C_q attestation)
`grant`	`ExecutionGrant`	Grant Authority	Runtime Admission / Enforcement Control
`ctd`	`ConsequentialToolDescriptor`	Resource Registry	Governance / Registry (CTD v1.0)
`hitl_witness`	`HitlWitness`	Evidence Provider	Evidence (verified-review witness)
`evidence`	`Evidence`	Evidence Provider	Evidence (telemetry / check)
`manifest`	`WitnessManifest`	Admission Controller	Runtime Admission (obligations)
`lease`	`AdmissionLease`	Admission Controller	Runtime Admission (pre-validation)
`saga_root`	`SagaExecutionRoot`	Grant Authority	Enforcement Control (saga budget)
`remediation`	`RemediationContract`	Execution Gateway	Execution (admission remediation)
`receipt`	`Receipt`	Execution Gateway / Resource Adapter / Outcome Service	Outcome Interface (receipt)
`delegation`	`DelegationRequest`	Agent Adapter	Runtime Admission (subagent scope)
`token_exchange`	`TokenExchangeBinding`	Execution Gateway	Enforcement Plane (RFC 8693)

Table 18: Thirteen envelope kinds in the proposed machine-readable profile in KF-JSON-0.2.

Ed25519 signatures use 86-character unpadded base64url encoding; the record and parameter digests below use lowercase hexadecimal SHA-256. JSON Web Key (JWK) thumbprints and WebAuthn challenges use their separately specified base64url forms:

$$\begin{aligned}
\text{signature} &= \text{base64url}\big(\text{Ed25519Sign}\big(sk, \text{UTF8}(\text{“KedgeFlow/KF-JSON-0.2”}) \\
&\qquad \| \text{ NUL } \| \text{ JCS}(E \setminus \{\text{signature}\})\big)\big) \\
H(q) &= \text{hex}\big(\text{SHA256}\big(D(\text{“proposal”}) \| \text{ NUL} \\
&\qquad \| \text{ JCS}(\{\text{schema} : E.\text{schema}, \text{payload} : E.\text{payload}\})\big)\big) \\
H(E) &= \text{hex}\big(\text{SHA256}\big(D(E.\text{kind}) \| \text{ NUL } \| \text{ JCS}(E)\big)\big) \quad (\forall E.\text{kind} \neq \text{“proposal”}) \\
H(\text{parameters}) &= \text{hex}\big(\text{SHA256}\big(D(\text{“parameters”}) \| \text{ NUL } \| \text{ JCS}(\text{parameters})\big)\big)
\end{aligned}$$

$H(q)$ excludes envelope signing metadata while preserving the exact payload. Changing subject, workload, DPoP key or delegation changes that payload and requires the continuity/revalidation rules of Section 6.6. Temporal fields are nonnegative UTC Unix milliseconds within $0 \ldots 2^{53} - 1$; the profile rejects fractional and unsafe numeric inputs.

D.3. Listings of Normative Fixtures

The `fixtures/` directory in the code package provides offline compatibility test vectors validating every envelope kind against demonstration keys and test data. Table 19 indexes the complete fixture inventory.

Fixture File	Envelope Kind	Signer Role	Profile Digest (abbreviated)
`proposal.json`	`proposal`	Agent Adapter	`3456dd204af1...252c8a3b`
`certificate.json`	`certificate`	Admission Controller	`28b208aceae2...5695db46`
`grant.json`	`grant`	Grant Authority	`c906936b8a7f...fc862fe4`
`ctd.json` (`ctd.yaml`)	`ctd`	Resource Registry	`ccdb4ed2ce5e...54757d6c`
`hitl_witness.json`	`hitl_witness`	Evidence Provider	`17be039938fa...80843e8c`
`evidence.json`	`evidence`	Evidence Provider	`dc427cc56de3...82862e16`
`manifest.json`	`manifest`	Admission Controller	`e725ae155d03...aa85cb5f`
`lease.json`	`lease`	Admission Controller	`e4b96e74051a...4fa02183`
`saga_root.json`	`saga_root`	Grant Authority	`02c9c7ac98db...d6a9b4cc`
`remediation.json`	`remediation`	Execution Gateway	`cc6fabe4828c...2f075aa6`
`receipt.json`	`receipt`	Resource Adapter	`afa7d9820c7d...d0011e23`
`delegation.json`	`delegation`	Agent Adapter	`bd13e0adf562...f2ec82af`
`token_exchange.json`	`token_exchange`	Execution Gateway	`86e6b2708f53...17db2070`

Table 19: Offline fixture inventory. Abbreviations are an index, not verification bytes; full values are in `fixture-digests.json`. The proposal row uses $H(q)$, and the other rows use the signed-record digest.

Supporting vectors in `fixtures/` include `public-test-keys.json` (authorized demonstration keys), `dpop-public-test-key.json` and `dpop-proof.jwt` (RFC 9449 proof vectors), `access-token.jwt` (OIDC/OAuth access token), `capacity-review-template.txt` and `capacity-review-rendered.txt` (WebAuthn review bytes), and `a2a-root-reference.json` (A2A protocol part). These are public demonstration vectors, including test-only review data; they grant no live authority and do not establish hardware enrollment. From the sibling package root, run the offline checks with `pytest tests/test_fixtures.py` or `python spec/validate.py`.

E. Glossary of Architectural Terms

Admission Certificate (C_q): A signed Admission Controller record that a particular proposal satisfied the declared obligation contract under specified policy and evidence. It is input to grant issuance, not execution authority.

Agent Plane: Existing runtimes, models, memory, planning, and unprivileged tool formulation. Its protected mutations enter KedgeFlow as proposals.

Execution Grant (G): A narrowly scoped, signed authorization bound to the proposal, principal, gateway audience, resource incarnation, validity, and execution allowance. Possessing the grant alone does not bypass caller or claim checks.

Complete Mediation: Every in-scope protected mutation path is enforced by the resource-side boundary. The deployment must verify its declared surface and expose exclusions.

Control Plane: Policy Authority, Resource Registry, Admission Controller, Grant Authority, and Outcome Service, with an interface to Evidence Providers. Logical roles need not be separate products or processes.

Enforcement Plane: Execution Gateway and trusted Resource Adapter, which redeem authority and invoke protected target operations.

Evidence-to-Authority Boundary: The transition from non-authoritative proposals and candidate evidence to policy admission and independently redeemable execution authority.

Execution Root: The durable authority record that owns a bounded execution allowance and links issuance, replacement, delegation, reservation, and redemption. New request IDs or nonces cannot multiply its allowance.

Claim: An atomic assignment of a grant's allowance to one attempt before dispatch. The claim remains consumed during outcome uncertainty.

Generation: The current authorization version of an execution root. Replacing a generation does not reset its budget.

Reservation: An authoritative allocation of shared capacity or budget to an action, with declared ownership and release conditions.

Fencing: Rejection of obsolete authority values at the declared ledger or target boundary. Preventing late target effects requires target enforcement.

Obligation and Witness: An obligation is a policy-required check; a witness is a scoped evidence record offered to discharge it. Admission decides whether the evidence is acceptable.

Level A Adapter: Atomically enforces its declared guard with target mutation. Its guarantee covers only the predicate and state scope participating in that atomic operation.

Level B Adapter: Revalidates state before an operation without atomic check-and-mutate. Concurrent changes can invalidate the checked predicate before the effect.

Level C Adapter: Dispatches an asynchronous operation and resolves its effect through authoritative outcome evidence. Reconciliation does not retroactively establish commit-time prevention.

Completion Mode: Dispatch-bound or commit-bound validity requirements. This choice is distinct from adapter atomicity and requires support at the chosen enforcement point.

Component and Contract Map: A classification of tools by supplied responsibility and privilege placement. Missing owners or incomplete interfaces expose gaps for a declared deployment.

Document Status

This document proposes a reference architecture and interface contract for consequential-action control planes. Component names and conformance requirements are proposals for discussion, not an adopted industry standard. Conditional properties follow from explicit trust, coverage, and resource assumptions; machine-readable schemas, signed compatibility fixtures, and bounded reference services accompany the source. The verification report distinguishes executed local checks from remaining deployment gates. No cross-vendor interoperability, production certification, commercial benchmark, or comparative efficacy result is claimed. AI-assisted drafting and review have been used; the authors are responsible for validating the final architectural and technical claims.

Version 0.9 is frozen on 6 October 2026. The frozen source, PDF and contract hashes are indexed in `archive/whitepaper-v0.9/FREEZE.json`. Implementation development proceeds in the Rust repository against this baseline; changes to the paper or its protocol contracts require a new version.

References

[1] Scott Rose, Oliver Borchert, Stu Mitchell, and Sean Connelly. Zero trust architecture. Technical Report SP 800-207, National Institute of Standards and Technology, 2020.

[2] Mark S. Miller, Ka-Ping Yee, and Jonathan Shapiro. Capability myths demolished. Technical Report SRL2003-02, Johns Hopkins University Systems Research Laboratory, 2003.

[3] LangChain. Langgraph: Interrupts. https://docs.langchain.com/oss/python/langgraph/interrupts, 2026. Documentation accessed 2026-10-02.

[4] Open Networking Foundation. OpenFlow switch specification, version 1.5.1. Technical report, Open Networking Foundation, 2015.

[5] Jun He and Deying Yu. Cognitive admission control: Risk-conditioned assurance for consequential actions in agentic distributed systems. *arXiv preprint arXiv:2609.16313*, 2026. https://arxiv.org/abs/2609.16313.

[6] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.

[7] Jun He and Deying Yu. Before agents act: Assurance-aware semantic scheduling for evidence acquisition in distributed systems. *arXiv preprint arXiv:2609.34376*, 2026. https://arxiv.org/abs/2609.34376.

[8] World Wide Web Consortium. Web authentication: An api for accessing public key credentials—level 3. W3C Recommendation, 25 August 2026. https://www.w3.org/TR/2026/REC-webauthn-3-20260825/, 2026. Recommendation accessed 2026-10-02.

[9] OpenAI. Openai agents sdk: Guardrails and tracing. https://openai.github.io/openai-agents-python/guardrails/; https://openai.github.io/openai-agents-python/tracing/, 2026. Documentation accessed 2026-10-02.

[10] Anthropic. Claude agent sdk: Configure permissions. https://code.claude.com/docs/en/agent-sdk/permissions, 2026. Documentation accessed 2026-10-02.

[11] Google. Agent development kit: Types of callbacks. https://adk.dev/callbacks/types-of-callbacks/, 2026. Documentation accessed 2026-10-02.

[12] LangChain. Langgraph overview. https://docs.langchain.com/oss/python/langgraph/overview, 2026. Documentation accessed 2026-10-02.

[13] Microsoft. Autogen documentation. https://microsoft.github.io/autogen/stable/, 2026. Documentation accessed 2026-10-02.

[14] Microsoft. Microsoft agent framework overview. https://learn.microsoft.com/en-us/agent-framework/overview/, 2026. Documentation accessed 2026-10-02.

[15] Microsoft. Microsoft agent framework: Agent middleware. https://learn.microsoft.com/en-us/agent-framework/concepts/agents/middleware/, 2026. Documentation accessed 2026-10-02.

[16] CrewAI. Crewai documentation: Flows. https://docs.crewai.com/en/concepts/flows, 2026. Documentation accessed 2026-10-02.

[17] CrewAI. Crewai documentation: Human-in-the-loop workflows. https://docs.crewai.com/en/learn/human-in-the-loop, 2026. Documentation accessed 2026-10-02.

[18] Model Context Protocol. Model context protocol specification: Tools, revision 2026-07-28. https://modelcontextprotocol.io/specification/2026-07-28/server/tools, 2026. Specification accessed 2026-10-02.

[19] Model Context Protocol. Model context protocol specification: Authorization, revision 2026-07-28. https://modelcontextprotocol.io/specification/2026-07-28/basic/authorization, 2026. Specification accessed 2026-10-02.

[20] A2A Protocol Working Group. Agent2agent protocol specification, version 1.0.0. https://a2a-protocol.org/latest/specification/, 2026. Specification accessed 2026-10-02.

[21] Open Policy Agent. Open policy agent documentation. https://www.openpolicyagent.org/docs, 2026. Documentation accessed 2026-10-02.

[22] Open Policy Agent. Open policy agent documentation: Bundles. https://www.openpolicyagent.org/docs/management-bundles, 2026. Documentation accessed 2026-10-02.

[23] Cedar. Cedar policy language reference guide: How cedar authorization works. https://docs.cedarpolicy.com/auth/authorization.html, 2026. Documentation accessed 2026-10-02.

[24] Cedar. Cedar policy language reference guide: Policy validation against schema. https://docs.cedarpolicy.com/policies/validation.html, 2026. Documentation accessed 2026-10-02.

[25] Nat Sakimura, John Bradley, Michael B. Jones, Breno de Medeiros, and Chuck Mortimore. Openid connect core 1.0 incorporating errata set 2. OpenID Foundation. https://openid.net/specs/openid-connect-core-1_0.html, 2023. Specification accessed 2026-10-02.

[26] SPIFFE Project. Spiffe concepts: Ids, trust domains, verifiable identity documents, and the workload api. https://spiffe.io/docs/latest/spiffe-about/spiffe-concepts/, 2026. Documentation accessed 2026-10-02.

[27] SPIFFE Project. Spire concepts: Architecture and workload attestation. https://spiffe.io/docs/latest/spire-about/spire-concepts/, 2026. Documentation accessed 2026-10-02.

[28] Daniel Fett, Brian Campbell, John Bradley, Torsten Lodderstedt, Michael B. Jones, and David Waite. RFC 9449: OAuth 2.0 Demonstrating Proof of Possession (DPoP). RFC Editor. https://www.rfc-editor.org/rfc/rfc9449, 2023.

[29] Michael B. Jones, Anthony Nadalin, Brian Campbell, John Bradley, and Chuck Mortimore. RFC 8693: OAuth 2.0 Token Exchange. RFC Editor. https://www.rfc-editor.org/rfc/rfc8693, 2020.

[30] etcd Authors. etcd documentation: Api, version 3.6. https://etcd.io/docs/v3.6/learning/api/, 2026. Documentation accessed 2026-10-02.

[31] etcd Authors. etcd documentation: Api guarantees, version 3.6. https://etcd.io/docs/v3.6/learning/api_guarantees/, 2026. Documentation accessed 2026-10-02.

[32] Temporal Technologies. Temporal workflow execution overview. https://docs.temporal.io/workflow-execution, 2026. Documentation accessed 2026-10-02.

[33] Temporal Technologies. What is a temporal activity? https://docs.temporal.io/activities, 2026. Documentation accessed 2026-10-02.

[34] Emily Fortuna. Saga compensating actions design pattern. Temporal Technologies. https://temporal.io/blog/compensating-actions-part-of-a-complete-breakfast-with-sagas, 2023. Official technical article accessed 2026-10-02.

[35] Envoy Project. Envoy documentation: External authorization. https://www.envoyproxy.io/docs/envoy/latest/configuration/http/http_filters/ext_authz_filter, 2026. Documentation accessed 2026-10-02.

[36] Kubernetes Authors. Kubernetes documentation: Api concepts. https://kubernetes.io/docs/reference/using-api/api-concepts/, 2026. Documentation accessed 2026-10-02.

[37] OpenTelemetry Authors. What is opentelemetry? https://opentelemetry.io/docs/what-is-opentelemetry/, 2026. Documentation accessed 2026-10-02.

[38] gVisor Authors. What is gvisor? https://gvisor.dev/docs/, 2026. Documentation accessed 2026-10-02.

[39] Microsoft. Agent control specification. https://github.com/microsoft/agent-governance-toolkit/blob/main/policy-engine/spec/SPECIFICATION.md, 2026. Draft specification accessed 2026-10-02.

[40] Tihan-Nico Paxton. Agent infrastructure control protocol. Technical Report draft-paxton-aicp-00, IETF, 2026. Internet-Draft, work in progress; accessed 2026-10-02. https://www.ietf.org/archive/id/draft-paxton-aicp-00.html.

[41] Zexun Wang. Proof-carrying agent actions: Model-agnostic runtime governance for heterogeneous agent systems. arXiv:2606.04104. https://arxiv.org/abs/2606.04104, 2026.

[42] Zexun Wang. Cava: Canonical action verification and attestation for runtime governance of agentic ai systems. arXiv:2607.13716. https://arxiv.org/abs/2607.13716, 2026.

[43] Marcelo Fernandez. Agent control protocol: Admission control for agent actions. *arXiv preprint arXiv:2603.18829*, 2026. https://arxiv.org/abs/2603.18829.

[44] Model Context Protocol. Model context protocol: Base protocol and request metadata, revision 2026-07-28. https://modelcontextprotocol.io/specification/2026-07-28/basic/index, 2026. Specification accessed 2026-10-02.

[45] Model Context Protocol. Model context protocol: Versioning and compatibility, revision 2026-07-28. https://modelcontextprotocol.io/specification/2026-07-28/basic/versioning, 2026. Specification accessed 2026-10-02.

[46] Amazon Web Services. Aws security token service api reference: Assumerole and assumerolewithwebidentity. https://docs.aws.amazon.com/STS/latest/APIReference/API_AssumeRole.html; https://docs.aws.amazon.com/STS/latest/APIReference/API_AssumeRoleWithWebIdentity.html, 2026. Documentation accessed 2026-10-02.

[47] Hector Garcia-Molina and Kenneth Salem. Sagas. Technical Report CS-TR-070-87, Princeton University, Department of Computer Science, January 1987. https://www.cs.princeton.edu/research/techreps/598.

[48] Anders Rundgren, Bradley Jordan, and Samuel Erdtman. RFC 8785: JSON Canonicalization Scheme (JCS). RFC Editor, 2020.

[49] Donald Eastlake 3rd and Tony Hansen. RFC 6234: US Secure Hash Algorithms (SHA and SHA-based HMAC and HKDF). RFC Editor, 2011.

[50] Simon Josefsson and Ilari Liusvaara. RFC 8032: Edwards-Curve Digital Signature Algorithm (EdDSA). RFC Editor, 2017.

[51] Dick Hardt. The oauth 2.0 authorization framework. RFC 6749, Section 5.1. https://www.rfc-editor.org/rfc/rfc6749, 2012. Accessed 2026-10-05.

[52] NGINX. Http core module: Large client header buffers. https://nginx.org/en/docs/http/ngx_http_core_module.html#large_client_header_buffers, 2026. Documentation accessed 2026-10-05.

[53] Amazon Web Services. Quotas for application load balancers: Http header limits. https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-limits.html, 2026. Documentation accessed 2026-10-05.

[54] Cloudflare. Increased http header size limit to 128 kb. https://developers.cloudflare.com/changelog/post/2025-10-16-header-limit-increase/, 2025. Published 2025-10-16; accessed 2026-10-05.

[55] etcd Authors. etcd documentation: Performance, version 3.6. https://etcd.io/docs/v3.6/op-guide/performance/, 2026. Documentation accessed 2026-10-02.

[56] Anthropic. Claude code: Configure permissions. https://code.claude.com/docs/en/permissions, 2026. Documentation accessed 2026-10-03.

[57] OpenAI. Codex: Permissions and sandboxing. https://learn.chatgpt.com/docs/permissions; https://learn.chatgpt.com/docs/sandboxing, 2026. Documentation accessed 2026-10-03.

[58] OpenAI. Sandbox security: Third-party credentials and trusted proxies. https://developers.openai.com/api/docs/guides/agents-api/environments/security, 2026. Documentation accessed 2026-10-03.

[59] Apple. Keychain access: Application access to keychain items. https://support.apple.com/guide/keychain-access/if-youre-asked-for-access-to-your-keychain-kyca1243/mac, 2026. Documentation accessed 2026-10-03.

[60] Docker. Protect the docker daemon socket. https://docs.docker.com/engine/security/protect-access/, 2026. Documentation accessed 2026-10-03.

[61] Apple. Configuring the macos app sandbox. https://developer.apple.com/documentation/xcode/configuring-the-macos-app-sandbox, 2026. Documentation accessed 2026-10-05.

[62] bubblewrap Contributors. Bubblewrap: Sandbox security and limitations. https://github.com/containers/bubblewrap/blob/main/README.md, 2026. Documentation accessed 2026-10-05.

[63] Linux Kernel Authors. Control group v2. https://www.kernel.org/doc/html/latest/admin-guide/cgroup-v2.html, 2026. Documentation accessed 2026-10-05.

[64] Docker. Rootless mode. https://docs.docker.com/engine/security/rootless/, 2026. Documentation accessed 2026-10-05.

[65] Docker. Bind mounts: Considerations and constraints. https://docs.docker.com/engine/storage/bind-mounts/, 2026. Documentation accessed 2026-10-05.

[66] Model Context Protocol. Model context protocol: Streamable http, revision 2026-07-28. https://modelcontextprotocol.io/specification/2026-07-28/basic/transports/streamable-http, 2026. Specification accessed 2026-10-02.

[67] Git Project. Git status documentation: Background refresh. https://git-scm.com/docs/git-status, 2026. Documentation accessed 2026-10-03.

[68] Git Project. Git configuration: Filesystem monitor hooks. https://git-scm.com/docs/git-config/2.43.1, 2026. Documentation accessed 2026-10-03.

[69] Kubernetes Authors. Kubernetes documentation: Safely drain a node. https://kubernetes.io/docs/tasks/administer-cluster/safely-drain-node/, 2026. Documentation accessed 2026-10-02.

[70] Kubernetes Authors. Kubernetes documentation: Api-initiated eviction. https://kubernetes.io/docs/concepts/scheduling-eviction/api-eviction/, 2026. Documentation accessed 2026-10-02.

[71] Kubernetes Authors. Admission control in kubernetes. https://kubernetes.io/docs/reference/access-authn-authz/admission-controllers/, 2026. Documentation accessed 2026-10-04.

[72] Istio Authors. Istio architecture: Control and data planes. https://istio.io/latest/docs/ops/deployment/architecture/, 2026. Documentation accessed 2026-10-04.